УНИВЕРЗИТЕТ У БЕОГРАДУ

ФАКУЛТЕТ ОРГАНИЗАЦИОНИХ НАУКА

ЗАВРШНИ РАД

Евалуација агената заснованих на великим језичким моделима при решавању проблема информационог инжењерства

Ментор

Др Андрија Петровић,

Доцент

Студент

Алекса Љујић, 2021/0276

Београд, 2026. године

Овај рад посвећујем својим родитељима, сестрама, теткама, браћи од тетака, баки и деди, чичи и мојој Кристини.

Хвала вам што сте током свих ових година били уз мене, пружали ми подршку, веровали у мене и били део мог образовања, одрастања и животног и професионалног пута.

Овај рад је мали део захвалности за све што сте учинили за мене.

АПСТРАКТ

Развој великих језичких модела довео је до појаве агентских система који, поред генерисања текста и кода, могу самостално да користе алате и делују у софтверском окружењу. У оваквим системима скуп доступних алата представља важан део интерфејса између језичког модела и окружења, али није у потпуности јасно у којој мери појединачни алати доприносе успешности решавања задатака и да ли њихова употреба оправдава додатни трошак. Циљ овог рада је испитивање утицаја скупа алата на успешност, трошак и способност локализације грешке код агента за решавање проблема информационог инжењерства, при фиксираном језичком моделу. У ту сврху развијен је сопствени агентски систем заснован на петљи позивања алата, са извршавањем команди у изолованим Docker контејнерима. Експеримент је спроведен на свих 300 задатака из скупа SWE-bench Lite, у факторском дизајну 2 × 2, којим се независно испитује утицај алата за детерминистичку замену текста и групе алата за претрагу и читање датотека, док је алат за извршавање команди доступан у свим условима. Као примарне мере посматрају се стопа решених задатака, трошак по решеном задатку и стопа локализације грешке, док се додатно анализирају потрошња токена, број корака, време извршавања и коришћење појединачних алата. Поређење конфигурација заснива се на упареним статистичким поступцима, интервалима поверења и анализи неинфериорности. Посебна пажња посвећена је разликовању ефекта доступности алата од ефекта њиховог стварног коришћења. Комбинација љуске и алата за замену текста решила је 119 од 299 задатака (39,8%), а пун скуп алата 99 (33,1%), чиме је потврђена неинфериорност мањег скупа, уз интервал поверења разлике који у целини лежи изнад нуле. Алати за претрагу повећали су трошак по покретању, али нису побољшали локализацију грешке. Хипотеза да алат за замену текста смањује трошак по решеном задатку потврђена је гранично, смањењем трошка од 10,0%, при чему тај ефекат потиче искључиво из услова са алатима за претрагу, док у условима без њих алат трошак не мења, јер га претежно одређују улазни токени, а не величина измене. Закључује се да код малог модела већи скуп алата не даје бољег агента, а да најбољи однос успешности и трошка међу испитаним условима пружа минималан скуп који садржи љуску и наменски алат за измену датотека.

Кључне речи: велики језички модели, агенти, агентски системи, позивање алата, информационо инжењерство, SWE-bench Lite, аблациона анализа, локализација грешке, трошак, однос цене и перформанси.

САДРЖАЈ

1 Увод 1

1.1 Циљ рада 2

1.2 Методологија 2

1.3 Хипотезе 3

1.4 Структура рада 4

2 Релевантна литература 4

2.1 Темељи агената који користе алате 5

2.2 Бенчмаркови за генерисање кода и квалитет скупова података 5

2.3 Агентски системи за решавање проблема информационог инжењерства 6

2.4 Коришћење алата, контекст и ефикасност 7

2.5 Методологија евалуације и статистичка поузданост 8

2.6 Празнина у литератури 8

3 Теоријске основе 10

3.1 Велики језички модели 10

3.1.1 Језичко моделовање 10

3.1.2 Трансформер архитектура 11

3.1.3 Генерисање и понашање модела у пракси 12

3.1.4 Ограничења 12

3.2 Агенти 13

3.2.1 Појам агента 13

3.2.2 Позивање алата 13

3.2.3 Петља закључивања и деловања 13

3.2.4 Архитектуре агентских система 14

3.3 Агенти за извршавање кода 15

3.3.1 Окружење и акције 15

3.3.2 Интерфејс између агента и рачунара 15

3.3.3 Изолација и поновљивост 16

3.4 Евалуациони оквир 16

3.4.1 Бенчмаркови за код 16

3.4.2 SWE-bench 16

3.4.3 Метрике 17

3.4.4 Статистичко поређење конфигурација 18

4 Методологија 19

4.1 Архитектура агента 20

4.2 Дизајн алата 23

4.2.1 Алат bash 24

4.2.2 Алат str_replace 25

4.2.3 Алат search 26

4.2.4 Алат read_file 27

4.2.5 Алат find_file 27

4.3 Окружење за извршавање 28

4.4 Поступак евалуације 29

4.4.1 Пилот евалуација 29

4.4.2 Скуп података 30

4.4.3 Дизајн експеримента 30

4.4.4 Ток једног покретања 31

4.4.5 Мере 34

4.4.6 Правила искључивања 34

4.4.7 Статистичка анализа 35

5 Опис и резултати експеримента 36

5.1 Поставка експеримента 36

5.2 Резултати 37

5.2.1 Провера хипотезе X1 37

5.2.2 Мера локализације 40

5.2.3 Провера хипотезе X2 42

5.2.4 Провера хипотезе X3 43

5.2.5 Утицај дефекта у алату `search` 45

5.3 Дискусија резултата 46

5.4 Ограничења 52

6 Закључак 57

6.1 Главни налази 58

6.2 Тумачење 58

6.3 Практичне препоруке 59

6.4 Ограничења 59

6.5 Даљи рад 60

Употреба алата заснованих на вештачкој интелигенцији 61

Литература 61

1 УВОД

Велики језички модели су у информационом инжењерству најпре коришћени за допуњавање кода у развојном окружењу и као асистенти у облику разговора. У таквом начину рада контекст држи човек. Он бира који део кода ће моделу показати, преноси предлог решења у пројекат, покреће програм и проверава да ли измена заиста решава проблем. Модел даје предлог, али не делује у окружењу.

Последњих година тежиште се помера ка вештачкој интелигенцији као агенту. Агент је систем у коме језички модел у петљи сам бира наредни корак: чита датотеке репозиторијума, покреће команде, мења код и проверава резултат, све док задатак не буде решен. Истраживачки системи као што су SWE-agent (J. Yang и др, 2024a) и OpenHands (Wang и др, 2025), као и комерцијални алати засновани на истом принципу, показују да агенти могу да решавају стварне проблеме из репозиторијума отвореног кода. Скуп података SWE-bench (Jimenez и др, 2024) постао је стандардна мера напретка у овој области.

Успешност агента, међутим, не зависи само од модела. Систем који окружује модел (енг. *harness*) одређује које алате модел има на располагању, како су описани и какве повратне информације добија. J. Yang и др (2024a) показују да дизајн интерфејса између агента и рачунара значајно утиче на резултат, иако је модел исти. Сваки алат ипак има и цену: његов опис се шаље моделу у сваком позиву, а резултати позива продужавају контекст, који се у наредним корацима поново обрађује и плаћа. Истовремено, Xia и др (2025) показују да се конкурентни резултати могу постићи и знатно једноставнијим системом, што доводи у питање да ли је сложенији скуп алата заиста потребан. Изоловани допринос појединачних алата мери се контролисаним аблацијама, али се оне углавном своде на уклањање једне компоненте у једном тренутку и на успешност као јединu меру, па изостаје поређење ефеката различитих група алата међу собом и њихов однос према трошку.

Питање трошка има практичну тежину. Коришћење модела путем API-ја плаћа се по токену, а агент у једном задатку прави десетине позива модела. Мали и јефтини модели су посебно занимљиви за примену, али је управо код њих отворено колико

добро користе алате које им систем пружа. Овај рад се бави тим питањем на примеру малог модела и стварних задатака из SWE-bench скупа.

1.1 ЦИЉ РАДА

Општи циљ рада је да се измери како скуп алата доступних агенту утиче на успешност решавања задатака из информационог инжењерства и на трошак тог решавања, при фиксираном језичком моделу.

Конкретни циљеви рада су:

- имплементирати сопствени агентски систем са скупом алата који се може конфигурисати и изолованим окружењем за извршавање;
- спровести аблацију скупа алата у факторском дизајну 2 × 2: присуство алата за замену текста (`str_replace`) и присуство групе алата за претрагу и читање датотека, уз љуску (`bash`) као основни алат у свим условима;
- измерити стопу решених задатака званичном SWE-bench процедуром евалуације, трошак по решеном задатку, као и способност агента да пронађе датотеку у којој се налази грешка;
- утврдити која конфигурација алата пружа најбољи однос цене и перформанси, а која доноси предност при лоцирању грешке;
- утврдити у којој мери агент заиста користи алате који су му доступни, како би се ефекат доступности алата разликовао од ефекта његовог коришћења.

1.2 МЕТОДОЛОГИЈА

За потребе рада развијен је сопствени агентски систем у програмском језику Python, чији је изворни код јавно доступан у репозиторијуму https://github.com/aleksaljujic/agent-harness. Агент ради у петљи позивања алата, а свака команда се извршава у изолованом Docker контејнеру без приступа мрежи, тако да агент не може да дође до званичне исправке проблема. Као модел се користи `gpt-5.4-nano`, са температуром 0 и искљученим режимом проширеног закључивања. Модел је намерно фиксиран, јер је предмет рада скуп алата, а не поређење модела.

Експерименту је претходила пилот евалуација на мањем, синтетички направљеном

скупу од 12 задатака опште намене (креирање и измена датотека, исправљање грешака, рад са тестовима). У пилоту је комбинација `bash` и `str_replace` остварила најнижи трошак и најкраће време извршавања, уз 11 решених задатака од 12. Пошто је пилот рађен на малом броју задатака, са по једним покретањем и на вештачки направљеним задацима, тај налаз није довољан за закључак. Он је, међутим, мотивисао систематски експеримент на стварним задацима, описан у наставку.

Главни експеримент спроводи се на целом скупу SWE-bench Lite (300 задатака), у четири конфигурације алата, што даје 1200 покретања. Све конфигурације користе идентичан системски упит, који не упућује на коришћење ниједног од алата који се пореде, тако да се између услова разликује само скуп доступних алата. Решења се оцењују званичним SWE-bench тестовима. Примарне мере су стопа решених задатака, трошак по решеном задатку и стопа локализације, тј. удео покретања у којима исправка агента мења бар једну датотеку коју мења и референтна исправка. Секундарне мере су трошак по покретању, број токена, број корака и време извршавања. Пошто сваки задатак пролази кроз све конфигурације, поређења су упарена. Мере, правила искључивања и конфигурација записани су пре покретања експеримента, а хипотезе и план статистичке анализе пре оцењивања решења и пре рачунања стопе локализације. Свако покретање бележи верзију кода којом је добијено. Детаљи су описани у поглављу 4.

1.3 ХИПОТЕЗЕ

Полазна теза рада је да комбинација љуске и алата за замену текста (`bash` и `str_replace`) пружа најбољи однос цене и перформанси, док пун скуп алата, који укључује и алате за претрагу и читање датотека, доноси предност пре свега при лоцирању грешке. Да би се ова теза могла проверити, рашчлањена је на три хипотезе:

- **X1:** Доступност алата за детерминистичку замену текста (`str_replace`) смањује трошак по решеном задатку;
- **X2:** Доступност алата за претрагу и читање датотека повећава трошак по покретању, али повећава стопу локализације, тј. удео покретања у којима агент измени датотеку у којој се налази грешка;
- **X3:** Стопа решених задатака конфигурације са алатима `bash` и `str_replace`

није нижа од стопе конфигурације са пуним скупом алата за више од пет процентних поена.

Хипотезе X1 и X2 проверавају се на главним ефектима факторског дизајна: ефекат алата `str_replace` добија се поређењем свих конфигурација са тим алатом и без њега, при оба нивоа другог фактора, а ефекат алата за претрагу и читање на исти начин. Хипотеза X3 је хипотеза неинфериорности и служи као контрола да уштеда из X1 није постигнута на рачун лошијих резултата. Ако се потврде све три хипотезе, из њих следи полазна теза: конфигурација са алатима `bash` и `str_replace` је најисплативији избор, а алати за претрагу и читање имају вредност при лоцирању грешке, али уз већи трошак. Разлике у ефекту између појединих врста задатака анализирају се експлоративно, без унапред формулисане хипотезе.

1.4 СТРУКТУРА РАДА

Рад је организован у шест поглавља. У другом поглављу дат је преглед литературе о агентима који користе алате, бенчмарковима за евалуацију генерисања кода и агентским системима за решавање проблема у репозиторијумима. Треће поглавље уводи теоријске основе: велике језичке моделе, појам агента и позивања алата, специфичности агената који раде над кодом и евалуациони оквир. Четврто поглавље описује методологију: архитектуру развијеног агента, дизајн алата и конфигурације које се пореде, окружење за извршавање и поступак евалуације. У петом поглављу приказани су поставка и резултати експеримента, њихова дискусија у односу на хипотезе и ограничења рада. Шесто поглавље сумира главне налазе и наводи правце даљег истраживања.

2 РЕЛЕВАНТНА ЛИТЕРАТУРА

Преглед литературе организован је тематски: најпре су приказани темељни радови о агентима који користе алате, затим бенчмаркови за евалуацију генерисања кода и њихова ограничења, агентски системи за решавање проблема у стварним репозиторијумима, радови о коришћењу алата и дугачког контекста, и на крају методолошки радови о статистички поузданој евалуацији.

2.1 ТЕМЕЉИ АГЕНАТА КОЈИ КОРИСТЕ АЛАТЕ

Парадигма ReAct (Yao и др, 2023) уводи наизменично смењивање корака закључивања и деловања, чиме језички модел може да планира, позива спољашње алате и прилагођава план на основу добијених опсервација. Ова петља представља основу већине савремених агентских система, укључујући и систем развијен у овом раду. Аутори је проверавају на четири задатка: одговарању на питања (HotpotQA), провери чињеница (Fever) и два задатка интерактивног одлучивања (ALFWorld и WebShop). На прва два задатка наизменично закључивање и деловање смањује измишљање чињеница, а на друга два ReAct надмашује методе засноване на опонашању и учењу поткрепљивањем за 34, односно 10 процентних поена апсолутне стопе успеха.

Паралелно, Schick и др (2023) показују да модел може самостално да научи када и како да позове алат (енг. *tool learning*), чиме се позивање алата успоставља као способност модела, а не само као спољашња надоградња. Модел се обучава самонадзирано, уз свега неколико примера по алату, и на тај начин користи калкулатор, систем за одговарање на питања, два претраживача, систем за превођење и календар. Резултат је знатно бољи учинак без претходних примера (енг. *zero-shot*), често на нивоу знатно већих модела, при чему се способност језичког моделовања не губи.

Оба рада показују да се коришћење алата може успоставити и измерити, али се њихове мере односе на задатке са малим бројем алата, у којима позив алата најчешће служи прибављању једне информације. Питање како састав скупа алата утиче на успешност и трошак агента који ради у стварном репозиторијуму остаје отворено и предмет је овог рада.

2.2 БЕНЧМАРКОВИ ЗА ГЕНЕРИСАЊЕ КОДА И КВАЛИТЕТ СКУПОВА ПОДАТАКА

Рана евалуација језичких модела над кодом ослањала се на кратке, самосталне задатке, као што су HumanEval (Chen и др, 2021) и MBPP (Austin, Odena и др, 2021). Каснији бенчмаркови проширују обухват ка сложенијим позивима библиотека (Zhuo и др, 2024) и ка евалуацији отпорној на контаминацију тренинг података (Jain и др, 2024). J. Liu и др (2023) показују да слаби тестови систематски прецењују перформансе модела,

што је директно релевантно за метрику коришћену у овом раду.

SWE-bench (Jimenez и др, 2024) помера евалуацију ка стварним проблемима (енг. *issues*) из GitHub репозиторијума, где модел мора да разуме постојећу кодну базу и произведе исправку (енг. *patch*), тј. скуп измена изворног кода који пролази задате тестове. Овај скуп података и његова евалуациона процедура представљају основу експерименталног дела рада. Проширења обухватају визуелне домене (J. Yang и др, 2024b) и више програмских језика (Zan и др, 2025). Истовремено, Aleithan и др (2024) документују озбиљне проблеме квалитета у SWE-bench-у: у 32,67% успешних исправки решење је било садржано у самом опису проблема или коментарима (енг. *solution leakage*), а 31,08% прихваћених исправки сумњиво је због тестова који нису довољни да провере исправност. Након уклањања проблематичних задатака, стопа решених задатака система SWE-agent са моделом GPT-4 пала је са 12,47% на 3,97%. Ови налази се разматрају у поглављу о ограничењима.

2.3 АГЕНТСКИ СИСТЕМИ ЗА РЕШАВАЊЕ ПРОБЛЕМА ИНФОРМАЦИОНОГ ИНЖЕЊЕРСТВА

J. Yang и др (2024a) уводе појам интерфејса између агента и рачунара (енг. *Agent-Computer Interface*, ACI) и показују да дизајн скупа алата значајно утиче на успешност агента. Аблацијом на SWE-bench Lite скупу пореде различите начине претраге, прегледа и измене датотека. Посебно је релевантно поређење са агентом који има само љуску: исти модел решава 18,00% задатака са прилагођеним интерфејсом, а 11,00% само са љуском. Ова аблација представља најближи претходни рад за хипотезе о утицају скупа алата. OpenHands (Wang и др, 2025) је отворена платформа опште намене за софтверске агенте; у овом раду је ипак развијен сопствени систем, како би се обезбедила потпуна контрола над скупом алата који се пореди. HyperAgent (Phan и др, 2024) је систем са четири специјализована агента (планер, навигатор, уређивач кода и извршилац). Аутори спроводе аблацију улога агената и анализу утицаја појединих алата, а уз стопу решених задатака извештавају и просечан трошак по задатку, што га чини методолошки најближим овом раду. Систем је проверен на више врста задатака: решавању проблема са GitHub-а, генерисању кода на нивоу репозиторијума и локализацији и исправљању дефеката. На скупу SWE-bench Lite

пуна конфигурација решава 26,00% задатака уз просечан трошак од 1,82 долара по задатку, док конфигурација у којој мање захтевне улоге преузима слабији модел решава 25,33% уз 0,45 долара. Разлика у успешности је, дакле, мала, а разлика у трошку вишеструка, што је налаз сродан питању које се поставља у овом раду. Међу осталим приступима издвајају се AutoCodeRover (Y. Zhang и др, 2024), који комбинује агента са техникама анализе програма, као и оквири за више агената попут AutoGen-a (Wu и др, 2023).

Посебно је важан рад Xia и др (2025), који доводи у питање потребу за сложеним агентима. Њихов приступ Agentless, заснован на трофазном процесу без агентске петље, у коме се редом локализује место грешке, генеришу измене и бира исправка на основу тестова, постиже 32,00% (96 тачних исправки) на SWE-bench Lite скупу уз просечан трошак од 0,70 долара, надмашујући тадашње агенте отвореног кода. Аутори такође конструишу SWE-bench Lite-S уклањањем проблематичних инстанци (Xia и др, 2025). Овај рад полази од сродног питања, али из другог угла: уместо да уклони агентску петљу, задржава је и мери да ли специјализовани алати доносе стварну корист у односу на рад само са љуском.

Други правац истраживања фокусира се на тренирање модела за решавање проблема у репозиторијумима: SWE-Fixer (Xie и др, 2025), SWE-Gym (Pan и др, 2025), SWE-RL (Y. Wei и др, 2025) и SWE-smith (J. Yang и др, 2025). Ти приступи мењају сам модел, кроз додатно обучавање или учење поткрепљивањем, па захтевају податке за обучавање и знатне рачунарске ресурсе, а резултат важи за модел који је на тај начин добијен. Овај рад полази са друге стране: модел остаје непромењен, а мења се само оно што га окружује, односно скуп алата. Два приступа су допунска, јер се налази о дизајну алата могу применити на било који модел, укључујући и оне посебно обучене за решавање проблема у репозиторијумима.

2.4 КОРИШЋЕЊЕ АЛАТА, КОНТЕКСТ И ЕФИКАСНОСТ

ToolLLM (Qin и др, 2023) и Gorilla (Patil и др, 2023) проучавају способност модела да ефикасно користе велики број API-ја, док τ-bench (Yao и др, 2024) евалуира агенте у интеракцији са алатима и корисником. N. F. Liu и др (2024) показују да модели слабије користе информације смештене у средину дугачког контекста, што објашњава

зашто више контекста не значи нужно боље резултате и подржава претпоставку да мањи скуп алата може бити ефикаснији. У истом духу, Xiao и др (2025) уклањају сувишне информације из трајекторије агента и тиме смањују број улазних токена за 39,9--59,7%, а укупан трошак за 21,1--35,9%, уз очуване перформансе.

2.5 МЕТОДОЛОГИЈА ЕВАЛУАЦИЈЕ И СТАТИСТИЧКА ПОУЗДАНОСТ

Bowman и Dahl (2021) разматрају системске проблеме бенчмаркова у обради природног језика. Card и др (2020) показују да је велики број експеримената у овој области статистички подмоћан и дају смернице за процену потребне величине узорка. Dror и др (2018) пружају практичан водич за избор статистичког теста, укључујући упарене тестове, а Reimers и Gurevych (2017) образлажу зашто треба извештавати дистрибуцију резултата уместо појединачне вредности: поређење две вредности добијене из по једног покретања може да произведе разлику која потиче само од варијабилности између покретања. Ови радови усмеравају статистичку обраду резултата у поглављу 4.

Смернице из ових радова примењене су непосредно. Пошто сваки задатак пролази кроз све четири конфигурације, поређења су упарена, па се за упарене бинарне исходе користи Мекнемаров тест, а за разлике у трошку по покретању Вилкоксонов тест ранга са знаком (Dror и др, 2018). Уместо појединачних вредности извештавају се интервали поверења добијени бутстрепом, а вишеструка поређења исправљају се Холмовом корекцијом. Величину узорка одређује сам скуп података, 300 задатака по конфигурацији, па се, у складу са Card и др (2020), при тумачењу води рачуна о томе да такав узорак поуздано открива само умерене разлике, док мале разлике остају у оквиру интервала поверења. Ограничење рада на једно покретање по услову, о коме упозорава Reimers и Gurevych (2017), наведено је у поглављу 5.

2.6 ПРАЗНИНА У ЛИТЕРАТУРИ

Претходни радови показују да дизајн скупа алата утиче на успешност агента, али то питање углавном обрађују на један од два начина. Први је поређење целих система, као у радовима SWE-agent (J. Yang и др, 2024a), OpenHands (Wang и др, 2025) и Agentless (Xia и др, 2025), где се истовремено разликују и алати, и упити, и организација рада

агента, па се допринос појединачних алата не може издвојити. Други је аблација у којој се компоненте уклањају једна по једна, као у радовима J. Yang и др (2024a) и Phan и др (2024), што показује да ли компонента помаже, али не и како се ефекти различитих компоненти међусобно односе.

Контролисане аблације саме површине алата, и то заједно са мером трошка, у новије време постоје. Lin и др (2026) испитују однос трошка и користи од извршавања кода при аутоматском поправљању програма, на 7 745 забележених трајекторија и 3 000 поновљених поправки, и налазе да ограничавање извршавања мало мења стопу успеха, а осетно смањује трошак. H. Yang и др (2026) пореде три површине алата (пун скуп, само љуску и само извршавање кода) при фиксираном моделу, систему и упиту, и налазе да површина мења пут и трошак, али не и стопу решених задатака. Оба рада, дакле, већ мере изоловани допринос групе алата и оба указују у истом смеру као и налази овог рада.

Оно што у њима изостаје јесте факторски нацрт. Површине алата пореде се једна по једна, па се не може проценити како се ефекат алата за измену и ефекат алата за претрагу међусобно односе, нити да ли један зависи од другог. Управо та зависност се овде показује као битна, јер алат за измену мења трошак само у присуству алата за претрагу. Факторски нацрт, у коме се две компоненте укрштају и посматра се и њихова интеракција, примењиван је на друге делове агентског система, на пример на планирање и памћење ранијих покушаја (J. Zhang и др, 2026), али, колико је аутору познато, не и на скуп алата. Поред тога, наведене аблације спроводе се са најјачим доступним моделима и готовим агентима, на пример Claude Code и OpenAI Codex CLI (H. Yang и др, 2026), док је мање познато колико успешно мали и јефтини модели решавају стварне задатке и колико им при томе помажу алати.

Овај рад доприноси у три правца. Главни допринос је примена факторског нацрта на скуп алата агента, односно систематско мерење ефекта две функционално различите групе: алата за измену кода (`str_replace`) и алата за претрагу и читање датотека. Групе се посматрају као два независна фактора у факторском дизајну 2 × 2, уз љуску као заједничку основу, при чему системски упит и сви остали услови остају исти. Тиме се ефекат сваке групе мери одвојено, при оба нивоа друге групе, а могуће је уочити и њихову међусобну интеракцију. Ефекат се не мери само на стопи решених задатака,

већ и на трошку по решеном задатку и на способности агента да пронађе датотеку у којој се налази грешка.

Споредни допринос рада је увид у то колико успешно мали модел решава стварне проблеме из информационог инжењерства. Експеримент се у целини спроводи са моделом `gpt-5.4-nano`, при температури 0 и без режима проширеног закључивања, на свих 300 задатака из скупа SWE-bench Lite. Резултати тако дају и основну меру могућности малог модела у агентском окружењу, као и одговор на питање колико му у томе помажу поједине групе алата.

Трећи допринос је методолошке природе и произашао је из тока самог истраживања. Поређење две верзије истог услова, пре и после исправке једног дефекта у алату за претрагу, показује да измерени „ефекат скупа алата“ обухвата и квалитет имплементације појединачног алата: исправка једног дефекта променила је исход једне хипотезе, при чему бројач грешака уграђен у сам алат ту грешку није бележио. Налаз је приказан у одељку 5.2.5 и говори у прилог томе да се грешке алата мере независно од тога како их алат сам пријављује.

3 ТЕОРИЈСКЕ ОСНОВЕ

Ово поглавље уводи појмове на којима почива остатак рада. Најпре су описани велики језички модели као основа система, затим појам агента и механизам позивања алата, који моделу омогућава да делује у окружењу. Након тога се разматрају специфичности агената који раде над изворним кодом, а поглавље се завршава описом евалуационог оквира: скупа података, метрика успешности и трошка, као и статистичких поступака за њихово поређење.

3.1 ВЕЛИКИ ЈЕЗИЧКИ МОДЕЛИ

3.1.1 Језичко моделовање

Језички модел дефинише расподелу вероватноћа над низовима токена, где је токен основна јединица текста (реч, део речи или симбол). Савремени модели су ауторегресивни: вероватноћа низа $x = (x_1, \ldots, x_T)$ разлаже се на производ условних

вероватноћа сваког токена у односу на претходне:

$$p_\theta(x) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}), \tag{1}$$

где θ означава параметре модела. Модел се тренира минимизацијом негативне лог-веродостојности над великим корпусом текста:

$$\mathscr{L}(\theta) = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}). \tag{2}$$

Дакле, основни задатак модела јесте предикција следећег токена. Сложеније способности, попут превођења, закључивања или писања кода, настају као последица тог једноставног циља примењеног на довољно велики модел и довољно велику количину података.

3.1.2 Трансформер архитектура

Готово сви савремени велики језички модели (енг. *Large Language Models*, LLM) засновани су на трансформер архитектури (Vaswani и др, 2017). Њен централни механизам је пажња (енг. *attention*), која сваком токену омогућава да директно приступи репрезентацијама свих претходних токена у контексту:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{3}$$

где су Q, K и V матрице упита, кључева и вредности добијене линеарним пројекцијама улаза, а d_k димензија кључева. Пошто се пажња рачуна између свих парова токена, рачунска и меморијска сложеност расту квадратно са дужином контекста. Због тога модели имају ограничен контекстни прозор, а обрада дугачких улаза је скупља и спорија.

3.1.3 Генерисање и понашање модела у пракси

При генерисању, модел у сваком кораку израчунава логите z за све токене речника, који се претварају у вероватноће уз параметар температуре τ:

$$p(x_t = i \mid x_{<t}) = \frac{\exp(z_i / \tau)}{\sum_j \exp(z_j / \tau)}. \tag{4}$$

Ниже вредности τ чине излаз детерминистичнијим, а више разноврснијим. Чак и при $\tau = 0$, излаз комерцијалних модела у пракси није потпуно детерминистички, што има директне последице по поновљивост експеримената.

Brown и др (2020) показују да довољно велики модели могу да решавају нове задатке само на основу инструкција и примера у улазу (енг. *in-context learning*), без промене параметара. Додатно подешавање модела да прати инструкције, укључујући учење на основу људских повратних информација (Ouyang и др, 2022), чини моделе употребљивим као асистенте опште намене. Навођење модела да пре коначног одговора експлицитно изложи међукораке закључивања (енг. *chain-of-thought*) значајно побољшава резултате на задацима који захтевају више корака (J. Wei и др, 2022).

3.1.4 Ограничења

Велики језички модели имају неколико ограничења која су релевантна за овај рад. Прво, знање модела је ограничено на податке из тренинга и не обухвата конкретну кодну базу над којом се ради, па модел мора да прикупи информације из окружења. Друго, модели могу да генеришу садржај који звучи уверљиво, али је нетачан (енг. *hallucination*) (Ji и др, 2023), нпр. да наведу непостојећу функцију или датотеку. Треће, иако контекстни прозори савремених модела обухватају стотине хиљада токена, модели не користе подједнако добро све делове контекста: N. F. Liu и др (2024) показују да се информације у средини дугачког улаза користе знатно слабије од оних на почетку и крају. Четврто, трошак коришћења модела путем API-ја обрачунава се по броју улазних и излазних токена, па свака додатна информација у контексту има директну цену.

3.2 АГЕНТИ

3.2.1 Појам агента

У класичној литератури вештачке интелигенције, агент је систем који опажа своје окружење путем сензора и делује на њега путем актуатора, са циљем да оствари задати циљ (Russell & Norvig, 2020). У контексту великих језичких модела, агент је систем у коме језички модел понавља циклус: на основу тренутног стања одлучује коју акцију да предузме, акција се извршава у окружењу, а резултат се враћа моделу као нова опсервација. За разлику од једноставног упита, где модел даје један одговор, агент самостално одређује број и редослед корака потребних за решавање задатка.

3.2.2 Позивање алата

Језички модел сам по себи производи само текст. Да би деловао у окружењу, користи алате (енг. *tools*): функције које извршава спољашњи систем, као што су читање датотеке, претрага или покретање команде. Schick и др (2023) показују да модел може да научи када и са којим аргументима да позове алат, а радови попут ToolLLM (Qin и др, 2023) и Gorilla (Patil и др, 2023) проширују ову способност на велики број стварних API-ја. Пошто је реч о наученој способности, на понашање модела утиче и то колико су назив и облик понуђеног алата слични онима на којима је модел обучаван.

У савременим системима, сваки алат се описује називом, описом на природном језику и шемом параметара (најчешће у JSON Schema формату). Опис свих доступних алата укључује се у контекст модела, а модел, уместо слободног текста, може да генерише структурисани позив алата са аргументима. Систем који покреће агента (енг. *harness*) извршава позив и резултат враћа моделу. Важна последица је да *сваки алат заузима део контекста*: описи алата се шаљу у сваком позиву модела, а већи број алата повећава и број могућих избора које модел мора да разликује.

3.2.3 Петља закључивања и деловања

Парадигма ReAct (Yao и др, 2023) комбинује закључивање и деловање: модел наизменично генерише мисао (образложење следећег корака) и акцију (позив алата),

након чега добија опсервацију. Формално, нека је q опис задатка. У кораку t контекст модела је

$$c_t = (q, r_1, a_1, o_1, \ldots, r_{t-1}, a_{t-1}, o_{t-1}), \tag{5}$$

где су r_i мисли, a_i акције, а o_i опсервације. Модел генерише наредну мисао и акцију $(r_t, a_t) \sim p_\theta(\cdot \mid c_t)$, окружење враћа опсервацију $o_t = \mathrm{Env}(a_t)$, и петља се наставља док модел не сигнализира крај или док се не достигне ограничење броја корака или трошка. Низ $(r_1, a_1, o_1, \ldots, r_n, a_n, o_n)$ назива се трајекторија агента.

Из ове формулације следи важно својство трошка. Пошто се у сваком кораку моделу шаље цео досадашњи контекст, укупан број улазних токена у трајекторији дужине n износи

$$N_{\text{in}} = \sum_{t=1}^{n} |c_t|, \tag{6}$$

што расте приближно квадратно са бројем корака ако свака опсервација додаје сличан број токена. Опширне опсервације (нпр. цео садржај великих датотека или дугачак излаз команди) и дугачки описи алата зато вишеструко повећавају трошак. Кеширање префикса упита (енг. *prompt caching*) код појединих провајдера смањује цену поновљених делова контекста, али не и њихов утицај на дужину контекста. Xiao и др (2025) показују да се скраћивањем трајекторија може значајно смањити трошак агентских система.

3.2.4 Архитектуре агентских система

Агентски системи се разликују по степену аутономије. На једном крају су фиксни процеси у којима програмер унапред одређује кораке, а модел решава појединачне подзадатке; пример је Agentless (Xia и др, 2025), који задатак дели на локализацију, поправку и валидацију. На другом крају су аутономни агенти, у којима модел сам бира наредну акцију, као у SWE-agent (J. Yang и др, 2024a) и OpenHands (Wang и др, 2025). Постоје и системи са више агената који имају различите улоге и међусобно комуницирају, као AutoGen (Wu и др, 2023) и HyperAgent (Phan и др, 2024).

Већа аутономија пружа флексибилност, али повећава број позива модела, трошак и варијабилност резултата.

3.3 АГЕНТИ ЗА ИЗВРШАВАЊЕ КОДА

3.3.1 Окружење и акције

Агенти за информационо инжењерство раде над стварним репозиторијумом кода. Њихово окружење чине систем датотека, љуска (енг. *shell*) и програмски алати пројекта, као што су интерпретер, систем за управљање зависностима и тестови. Типичне акције таквих агената могу се груписати у неколико категорија:

- **навигација и претрага** — листање директоријума, претрага датотека по имену и садржају (нпр. `find`, `grep`);
- **читање** — приказ садржаја датотеке или њеног дела;
- **измена** — креирање датотека и измена постојећег кода, нпр. заменом тачно одређеног дела текста;
- **извршавање** — покретање произвољних команди, скрипти за репродукцију грешке и тестова.

Произвољна команда љуске је у принципу довољна за све наведене акције, па се поставља питање да ли специјализовани алати доносе стварну предност или само повећавају контекст. Управо ово питање је предмет хипотеза овог рада.

3.3.2 Интерфејс између агента и рачунара

J. Yang и др (2024a) уводе појам интерфејса између агента и рачунара (ACI), по аналогији са корисничким интерфејсом за људе. Њихова теза је да језички модели имају другачије потребе од људи, па алати направљени за људе (нпр. интерактивни уређивачи текста) нису оптимални за агенте. Као принципе доброг интерфејса наводе једноставне акције са јасном документацијом, концизне и информативне повратне информације, као и заштитне механизме, нпр. проверу синтаксе након измене, који спречавају нагомилавање грешака. Аблацијом показују да дизајн интерфејса значајно утиче на успешност агента, иако је модел исти.

3.3.3 Изолација и поновљивост

Пошто агент извршава произвољне команде, окружење мора бити изоловано од рачунара на коме се систем покреће. Уобичајено решење су контејнери (нпр. Docker), који обезбеђују да сваки задатак почиње из истог, унапред дефинисаног стања, са инсталираним зависностима одговарајућих верзија. Изолација је истовремено безбедносни захтев и предуслов за поновљивост: разлике у окружењу не смеју да утичу на резултат поређења конфигурација.

3.4 ЕВАЛУАЦИОНИ ОКВИР

3.4.1 Бенчмаркови за код

Рани бенчмаркови, као што су HumanEval (Chen и др, 2021) и MBPP (Austin, Odena и др, 2021), састоје се од кратких, самосталних функција које модел генерише на основу описа, а исправност се проверава тестовима. За такве задатке уведена је метрика pass@k, вероватноћа да је бар једно од k генерисаних решења тачно. Ако се за задатак генерише $n \geq k$ решења, од којих је c тачних, непристрасна процена износи (Chen и др, 2021):

$$\text{pass@}k = \mathbb{E}_{\text{задаци}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]. \tag{7}$$

Такви задаци, међутим, не захтевају рад са постојећом кодном базом, што је суштина рада агената. Поред тога, J. Liu и др (2023) показују да недовољни тестови доводе до прецењивања перформанси, јер нетачна решења пролазе проверу.

3.4.2 SWE-bench

SWE-bench (Jimenez и др, 2024) састоји се од 2294 задатка прикупљених из стварних проблема и одговарајућих захтева за спајање (енг. *pull requests*) у 12 популарних Python репозиторијума. Сваки задатак садржи опис проблема и стање репозиторијума пре исправке. Систем треба да произведе исправку у облику разлике у односу на изворни код. Исправка се сматра успешном ако након њене примене:

- прођу тестови који су пре исправке падали, а које је увела оригинална исправка (FAIL_TO_PASS), и
- и даље пролазе тестови који су пролазили и пре исправке (PASS_TO_PASS), чиме се проверава да исправка није покварила постојећу функционалност.

Поред пуног скупа, често се користи SWE-bench Lite, подскуп од 300 тест задатака који смањује трошак евалуације. Задаци су изабрани филтрирањем пуног скупа: уклоњени су задаци са сликама, спољашњим линковима и референцама на друге захтеве, задаци чији опис има мање од 40 речи, као и задаци чија референтна исправка мења више од једне датотеке, има више од три измењена блока (енг. *hunks*) или креира и брише датотеке. Lite стога садржи самосталне, локализоване исправке грешака, што треба имати у виду при уопштавању резултата.

Приликом тумачења резултата треба имати у виду познате проблеме скупа. Aleithan и др (2024) показују да је у значајном делу успешних решења исправка већ садржана у опису проблема, као и да су поједини тестови превише слаби да открију нетачне исправке. Такође, пошто су репозиторијуми јавно доступни, постоји ризик да су задаци или њихова решења били део тренинг података модела (енг. *data contamination*), што је мотивисало бенчмаркове попут LiveCodeBench (Jain и др, 2024).

3.4.3 Метрике

Нека је $\mathcal{D}$ скуп од N задатака, а $y_i \in \{0, 1\}$ индикатор да је задатак i решен. Основна метрика успешности је стопа решених задатака (енг. *resolve rate*):

$$R = \frac{1}{N} \sum_{i=1}^{N} y_i. \tag{8}$$

Трошак задатка i одређује се на основу броја улазних и излазних токена свих позива модела у трајекторији и цене по токену коју одређује провајдер:

$$C_i = N_{\text{in}}^{(i)} \cdot p_{\text{in}} + N_{\text{out}}^{(i)} \cdot p_{\text{out}}, \tag{9}$$

где N_{in} следи из израза (6). Код провајдера који подржавају кеширање, улазни токени се додатно деле на кеширане и некеширане, са различитим ценама. Из трошка се изводе просечан трошак по задатку $\bar{C} = \frac{1}{N}\sum_i C_i$ и трошак по решеном задатку:

$$C_{\text{решен}} = \frac{\sum_{i=1}^{N} C_i}{\sum_{i=1}^{N} y_i}, \tag{10}$$

која обухвата и трошак неуспешних покушаја. Допунске метрике су број корака (позива модела) по задатку, укупан број токена и време извршавања.

Стопа решених задатака не показује да ли је агент бар пронашао место грешке. Зато се, по узору на Xia и др (2025), посматра и локализација на нивоу датотеке. Нека је $F_i^{\text{агент}}$ скуп датотека које мења исправка агента, а $F_i^{\text{реф}}$ скуп датотека које мења референтна исправка задатка i. Стопа локализације је

$$L = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}\left[F_i^{\text{агент}} \cap F_i^{\text{реф}} \neq \emptyset\right]. \tag{11}$$

Пошто у скупу SWE-bench Lite референтна исправка увек мења тачно једну датотеку, $F_i^{\text{реф}}$ има један елемент, па метрика мери да ли је агент изменио управо ту датотеку. Ова метрика даје информацију и о задацима које агент није успео да реши, што је посебно корисно код слабијих модела, код којих је стопа решених задатака ниска.

Пошто се конфигурације разликују и по успешности и по трошку, поређење се врши у дводимензионалном простору. Конфигурација A доминира конфигурацијом B ако није ни скупља ни мање успешна, а бар по једном критеријуму је строго боља. Скуп недоминираних конфигурација чини Парето фронт, који приказује најбоље доступне компромисе између цене и перформанси.

3.4.4 Статистичко поређење конфигурација

Резултати агената подложни су случајности, како због стохастичности модела, тако и због коначног броја задатака. Reimers и Gurevych (2017) показују да поређење заснивано на једном покретању може довести до погрешних закључака, па је пожељно извештавати расподелу резултата кроз више покретања. Card и др (2020) указују на то

да су многи експерименти у обради природног језика статистички подмоћни, тј. да број примера није довољан да се поуздано открије разлика очекиване величине.

Када се две конфигурације евалуирају на истим задацима, опажања су упарена, па треба користити упарене тестове (Dror и др, 2018). За бинарни исход (решен/нерешен) одговарајући је Мекнемаров тест (McNemar, 1947), који разматра само неслагања: нека је b број задатака које решава само конфигурација A, а c број задатака које решава само конфигурација B. Под нултом хипотезом да су конфигурације подједнако успешне, b има биномну расподелу $\mathrm{Bin}(b + c, 0{,}5)$, а за довољно велико $b + c$ користи се апроксимација

$$\chi^2 = \frac{(b - c)^2}{b + c}. \tag{12}$$

За непрекидне метрике, као што је трошак по покретању, могу се користити Вилкоксонов тест ранга са знаком или пермутациони тестови. Интервали поверења за стопу решених задатака и разлике између конфигурација могу се проценити бутстреп методом (Efron & Tibshirani, 1993), поновљеним узорковањем задатака са враћањем. Исти поступак се примењује и на трошак по решеном задатку, који је количник збирова и нема вредност по појединачном задатку, па се за њега не може директно применити упарени тест.

Неуспех статистичког теста да открије разлику не значи да разлика не постоји. Када је циљ показати да нова конфигурација није битно лошија од референтне, користи се тест неинфериорности: унапред се одређује маргина δ, највећа разлика која се сматра прихватљивом, а неинфериорност се закључује ако доња граница интервала поверења за разлику $R_A - R_B$ лежи изнад $-\delta$. На тај начин се тврдња „конфигурације су подједнако успешне“ замењује проверљивом тврдњом „конфигурација A није лошија од конфигурације B за више од δ“.

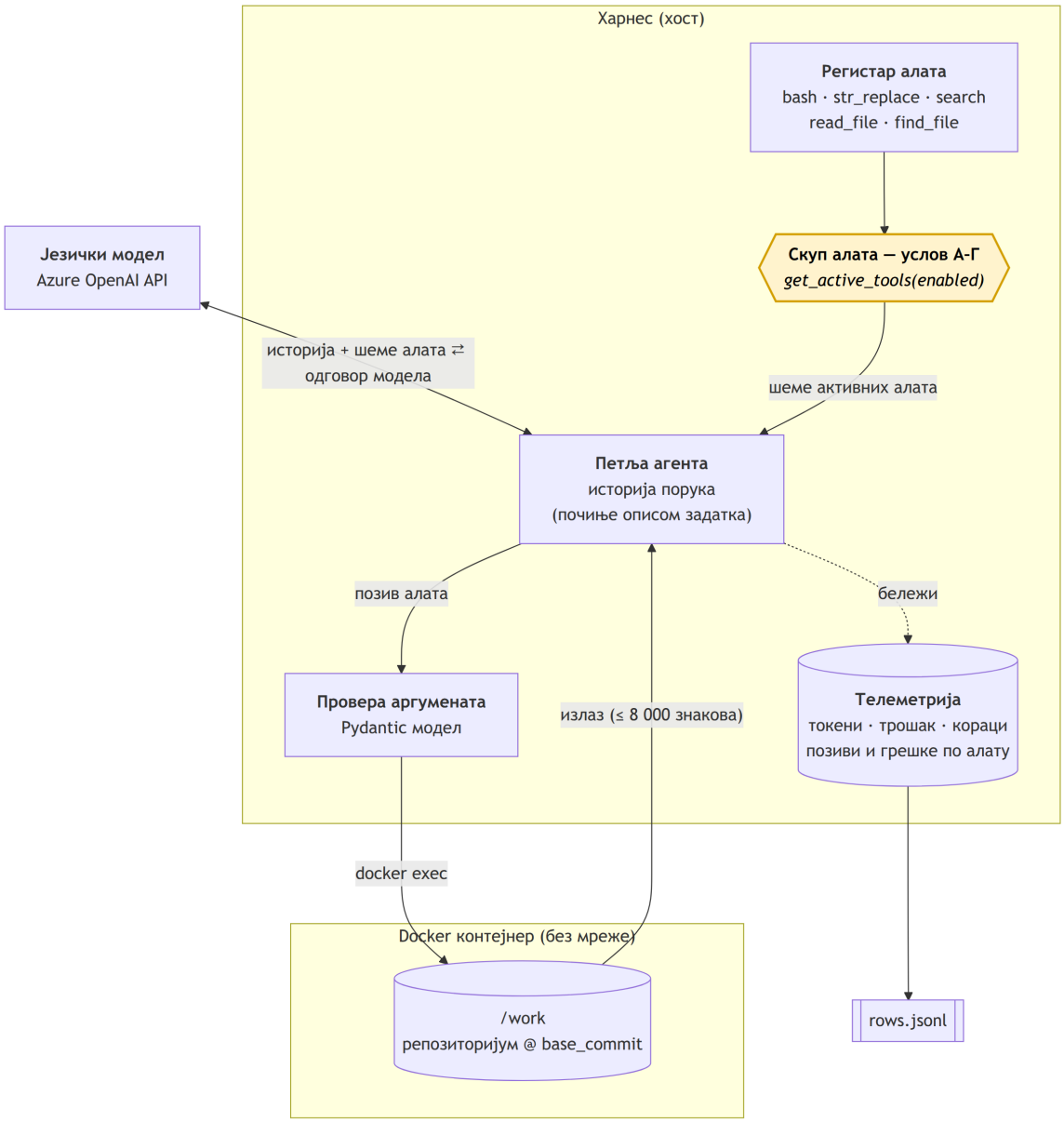
4 МЕТОДОЛОГИЈА

Ово поглавље описује систем развијен за потребе рада и поступак којим је спроведен експеримент. Најпре је приказана архитектура агента, затим алати који му могу бити доступни и начин на који се бирају, окружење у коме се команде извршавају и, на крају,

поступак евалуације: скуп података, дизајн експеримента, ток једног покретања, мере, правила искључивања и план статистичке анализе. Све вредности наведене у овом поглављу исте су у свим условима експеримента, осим скупа доступних алата.

4.1 АРХИТЕКТУРА АГЕНТА

Агент је реализован као петља позивања алата у духу парадигме ReAct (Yao и др, 2023). Компоненте система и ток података између њих приказани су на Слици 1.

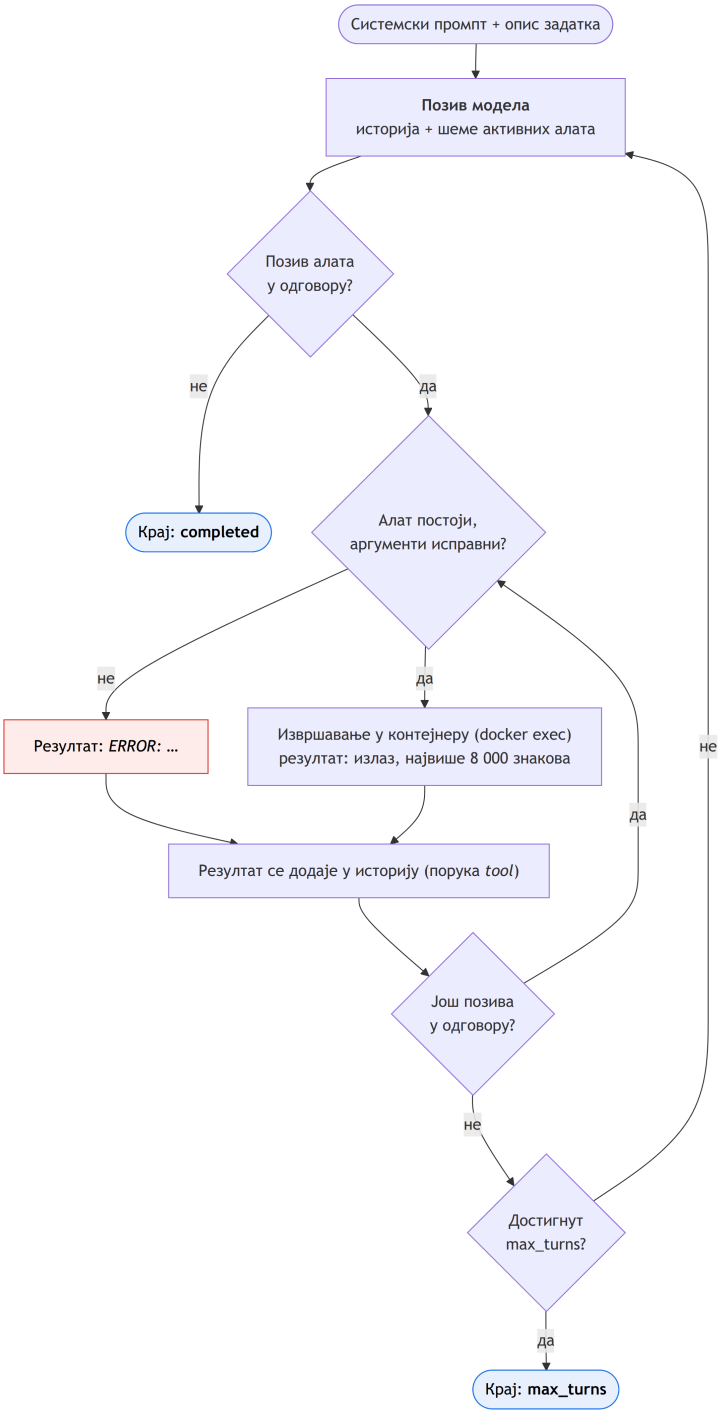


Слика 1. Архитектура развијеног агентског система

Један корак петље састоји се од следећих радњи:

1. Модел добија историју поруке (системски упит, опис задатка и све досадашње позиве алата са резултатима) и JSON шеме активних алата.
2. Ако одговор модела не садржи позив алата, покретање се завршава (разлог завршетка `completed`).
3. У супротном се аргументи сваког позива валидирају, позив се извршава у изолованом окружењу, а резултат се враћа моделу као порука улоге `tool`.
4. Петља се понавља највише `max_turns` корака, након чега се покретање прекида (разлог завршетка `max_turns`).

Модел ништа не извршава директно, већ само предлаже позив алата, а систем одлучује да ли ће га и како извршити. Грешке су део разговора: неисправни аргументи, непознат алат или неуспешна команда не прекидају петљу, већ се моделу враћају као текстуална порука облика `ERROR: ...`, на коју модел може да реагује. Излаз алата се тиме третира као део упита, па је јаснија порука о грешкама део дизајна система. Ток једног покретања кроз петљу, укључујући обраду грешака, приказан је на Слици 2.



Слика 2. Петља агента: позиви модела, извршавање алата и услови завршетка

Комуникација са моделом одвојена је од петље апстракцијом провајдера, која подржава OpenAI и Azure OpenAI API. Петља зна само за провајдерски неутралан облик одговора (текст, листа позива алата и потрошња токена), па се исти агент може користити са различитим провајдерима без измене логике.

Систем током рада бележи телеметрију: број улазних, излазних и токена закључивања, цену израчунату на основу ценовника модела, број позива модела, време проведено у позивима модела и у извршавању алата, као и број позива и грешака по алату. Провајдер повремено одбија исправан захтев због лажно позитивне класификације садржаја (`invalid_prompt`); такав захтев се понавља највише два пута, а свака друга грешка API-ја прекида покретање. Петља подржава и кооперативни сигнал за заустављање, који користи спољашње ограничење укупног трајања покретања.

Контекст се не сажима: историја поруке расте током целог покретања. Као што је показано у изразу (6), број улазних токена тада расте приближно квадратно са бројем корака, па је ограничавање величине излаза алата важан део контроле трошка.

4.2 ДИЗАЈН АЛАТА

Агент у експерименту има на располагању највише пет алата. Модел сваки алат види као опис функције са шемом параметара, а та шема се шаље уз сваки позив модела, без обзира на то да ли ће алат у том кораку бити употребљен. Пре извршавања систем проверава аргументе позива. Неисправни аргументи или непознат алат не прекидају рад: грешка се враћа моделу као обичан текстуални резултат, на који он може да реагује у следећем кораку.

Сви алати раде у истом изолованом контејнеру, у директоријуму `/work`, у коме се налази изворни код пројекта. Алати `bash` и `search` своде се на једну команду љуске, а `str_replace`, `read_file` и `find_file` у контејнеру покрећу кратку скрипту. Излаз сваког алата ограничен је на 8000 знакова, а дужи излаз се сече и на крај се додаје напомена да је приказ непотпун. Преглед алата дат је у Табели 1. Пуне JSON шеме свих алата, у облику у коме се шаљу моделу, доступне су у изворном коду система, у директоријуму `src/harness/tools` на грани `main` репозиторијума https://github.com/aleksaljujic/agent-harness.

Табела 1: Алати агента

Алат	Намена	Обавезни параметри	Опциони параметри	Ограничење излаза	Шема
`bash`	произвољна команда љуске	команда, образложење	---	8000 знакова, 60 s	37
`str_replace`	измена постојеће датотеке	путања, стари текст, нови текст	све појаве, редни број појаве	---	223
`search`	претрага садржаја по регуларном изразу	образац	филтер датотека	50 погодака	43
`read_file`	читање датотеке са бројевима линија	путања	почетна и завршна линија	8000 знакова	91
`find_file`	претрага датотека по имену	образац	највећи број резултата	100 резултата	105

Колона „Шема“ даје број улазних токена које опис алата додаје сваком позиву модела. Вредности су измерене на моделу `gpt-5.4-nano`, као разлика броја улазних токена истог захтева послатог са алатом и без њега. Кад је у листи бар један алат, томе се једном по позиву додаје још 98 токена за оквир листе. Услови из нацрта 2 × 2 тако носе по позиву: А 135, Б 358, В 374 и Г 597 токена. Сам `str_replace` (223) кошта скоро колико цела група `search`, `read_file` и `find_file` заједно (239), јер његов опис садржи упутства за случај вишеструког поклапања. Систем има и шести алат, `run_tests`, за покретање тестова са структурисаним резултатом, али он није био део експеримента, јер у овој поставци агент ради без покретања тестова пројекта.

4.2.1 Алат bash

Алат `bash` је основни алат и једини који је присутан у сваком услову. Сам по себи довољан је за решавање задатка: кроз њега агент може да претражује, чита и мења датотеке, али без икакве контроле над обликом и обимом излаза. Алат прима команду љуске и враћа спојен стандардни излаз и излаз за грешке.

Шема има два обавезна параметра: саму команду и поље за образложење. У то поље модел пре сваког позива кратко наводи зашто покреће команду. Систем образложење

не користи при извршавању, али оно остаје у историји разговора, па га модел види у наредним корацима, а сачувано је и у транскриптима покретања.

Свака команда извршава се као нов процес љуске у `/work`. Стање попут промене директоријума или променљивих окружења зато се не преноси из једног позива у други, па модел зависне кораке мора да споји у једну команду.

Излаз дужи од 8000 знакова се сече, а на крај се додаје напомена о томе колико је знакова изостављено, уз савет да се команда сузи, како модел непотпун приказ не би схватио као целину. Команда која траје дуже од 60 секунди се прекида, а модел добија поруку о истеку времена. Ако команда не произведе никакав излаз, враћа се само њен излазни код. Тако се празан резултат разликује од грешке, а модел зна да ли је тиха команда успела.

4.2.2 Алат str_replace

Алат `str_replace` мења постојећу датотеку тако што тачно наведени текст замени новим, без поновног генерисања целе датотеке. Обавезни параметри су путања, стари и нови текст. Поклапање је дословно, а не по регуларном изразу, па знакови који су у коду чести, попут тачке, звездице и заграда, не морају да се означавају.

Подразумевано, стари текст мора да се појави тачно једном. Ако га нема, модел добија грешку са саветом да прочита датотеку и препише текст заједно са увлачењем. Ако се појављује више пута, измена се не врши, а порука наводи број појава и број линије сваке од њих. Порука уз то саветује да се стари текст прошири навише, до претходног коментара или дефиниције функције или класе, јер су поновљени блокови кода обично једнаки и испод, па ширење наниже ретко разрешава двосмисленост. Ова провера је безбедносно својство алата, јер би без ње измена тихо погодила погрешно место.

За намерно вишеструке измене постоје два опциона параметра. Један мења све појаве, а други само одређену појаву по редоследу у датотеци, бројано од прве. Параметри се међусобно искључују, па њихова комбинација враћа грешку, а редни број ван опсега враћа грешку са стварним бројем појава. После успешне измене модел добија број замењених појава и линије на којима су биле.

Алат постоји зато што је ово једина операција коју `bash` ради лоше. Програм `sed` ради ред по ред, па не може да обухвати блок од више линија, а сваки посебан знак из кода мора да се означи. Над обрасцем који се јавља више пута `sed` тихо мења све појаве, без упозорења.

4.2.3 Алат search

Алат `search` претражује садржај датотека у целом пројекту по проширеном регуларном изразу. Обавезан параметар је образац. Опциони филтер има два значења: ако не садржи косу црту, тумачи се као образац имена датотеке, на пример све Python датотеке, а ако садржи косу црту, тумачи се као путања датотеке или директоријума на коју се претрага ограничава. Ова подела постоји зато што се филтер по имену пореди само са основним именом датотеке, па би путања као филтер тихо промашила све датотеке.

Сваки погодак враћа се у једном реду: путања, број линије и садржај линије. Тај облик непосредно је употребљив за `read_file` и `str_replace`. Директоријуми са историјом верзија (`.git`) и инсталираним зависностима (`node_modules`) се прескачу, јер у њима нема кода који агент треба да мења, а стварају много шума.

Алат враћа највише 50 погодака, редоследом обиласка система датотека, а не по релевантности. Када погодака има више, остали се одбацују без икакве назнаке: модел не добија ни укупан број погодака ни ознаку да је листа скраћена. По томе се `search` разликује од `find_file`. Уз то важи и општа граница од 8000 знакова, па дугачке линије могу да пресеку излаз и пре педесетог погодка, овај пут уз напомену. Када нема погодака, модел добија изричиту поруку уместо празног излаза, а неисправан регуларни израз враћа поруку о грешци.

Алат не даје агенту ништа што `bash` већ не може. Његова вредност је у ограничавању излаза, јер необуздана рекурзивна претрага над правим репозиторијумом може једним позивом да попуни контекст.

4.2.4 Алат read_file

Алат `read_file` чита датотеку и враћа њен садржај са бројевима линија. Обавезан параметар је путања, а опциони су почетна и завршна линија, које се броје од прве и улазе у опсег. Без њих се враћа цела датотека.

Сваки ред излаза почиње бројем линије и табулатором. Бројеви линија повезују алате: погоци из `search` и грешке из `str_replace` наводе линије, па агент може одмах да затражи тачан опсег. Број линије није део датотеке, па опис алата `str_replace` изричито упозорава да се тај префикс уклони пре него што се текст препише у стари текст, јер у супротном тражени текст неће бити пронађен.

Опсег се проверава пре читања. Грешку враћају почетна линија мања од прве, почетна линија после завршне и почетна линија после краја датотеке, при чему порука у последњем случају наводи и колико линија датотека има. Завршна линија после краја датотеке није грешка, него се тихо своди на последњу линију, јер је читање до краја уобичајена намера. У томе је разлика у односу на `bash`, где исечак датотеке са погрешним опсегом не враћа ништа осим излазног кода, док овде модел добија грешку по којој може да поступи.

Посебне поруке постоје и за непостојећу датотеку, за директоријум и за датотеку која није у кодирању UTF-8. Алат нема сопствено ограничење броја линија, па важи општа граница од 8000 знакова. Велика датотека прочитана без опсега се сече уз напомену, и агент тада треба да је чита по деловима.

4.2.5 Алат find_file

Алат `find_file` тражи датотеке по имену или путањи, а не по садржају, за шта служи `search`. Обавезан параметар је образац, а опциони параметар одређује највећи број резултата и подразумевано износи 100.

Ако образац садржи џокер знакове (звездицу, упитник или угласту заграду), тумачи се као глоб и пореди се и са именом датотеке и са њеном путањом у односу на корен пројекта. Тако раде и обрасци по имену, на пример све Python датотеке, и обрасци по путањи, на пример тестови у одређеном поддиректоријуму. Образац без џокер

знакова тумачи се као део имена датотеке, уз разликовање великих и малих слова. Враћају се само датотеке, не и директоријуми, а као и код `search`, прескачу се `.git` и `node_modules`.

Резултат је списак релативних путања, по једна у реду, абецедно уређен, па исти упит увек даје исти редослед. Када погодака има више од граничне вредности, приказује се првих N, а на крају се додаје ред са укупним бројем погодака, у облику „приказано првих N од M“. За разлику од `search`, модел тако зна да је списак непотпун и колико треба да сузи образац. Када погодака нема, модел добија изричиту поруку.

Као и `search`, овај алат не доноси нову могућност, јер исто може да се уради кроз `bash`. Разлика је у облику излаза: програм `find` има заставице које модел мора сваки пут да погоди, а облик и обим излаза зависе од њих, док `find_file` увек даје исти, ограничен и уређен облик резултата.

4.3 ОКРУЖЕЊЕ ЗА ИЗВРШАВАЊЕ

Свако покретање добија сопствени, једнократни Docker контејнер. Коришћена слика садржи Python 3.12 и основне алате, али не и зависности пројекта над којим агент ради. Агент зато не може да покрене тестове пројекта, а покретање кода пројекта често се завршава грешком због недостајућих модула. Из истог разлога алат `run_tests` није део експеримента.

Јединствена слика за све репозиторијуме изабрана је због упоредивости услова и изводљивости експеримента: окружење је исто за сваки задатак, па разлике у зависностима не могу да утичу на поређење услова, а 1200 покретања остаје у оквиру расположивог времена и средстава. Припрема засебног окружења по задатку захтевала би вишеструко више простора и времена.

Последица те одлуке је да агент не може да покрене тестове пројекта, а грешку може да репродукује само делимично: зависно од услова, између 34% и 54% покретања Python-а завршава се грешком због недостајућег модула, па се агент у великој мери ослања на читање кода и опис проблема. Успут се уклања и могућност да агент подешава исправку према постојећим тестовима, мада главну заштиту од доласка до званичног решења чине искључен приступ мрежи и уклоњена историја верзија, пошто

скривени тестови којима се исправка оцењује ионако нису део радне копије.

Контејнер се покреће са ограниченим животним веком, тако да се уклања и ако систем неочекивано прекине рад, а команде прима преко `docker exec`. Систем датотека је трајан између позива, али стање љуске није: свака команда почиње у новој љусци, што је наведено и у системском упиту. Радна копија репозиторијума повезана је са контејнером као директоријум `/work` (енг. *bind mount*), па су измене агента одмах видљиве на рачунару домаћину, одакле се издваја исправка.

Контејнер ради са следећим ограничењима:

- без приступа мрежи (`--network none`), па агент не може да преузме званичну исправку проблема;
- највише 2 GB меморије и 256 процеса;
- процеси раде под корисником домаћина, а не као `root`;
- скрипте алата доступне су само за читање;
- излаз команде ограничен је на 8000 знакова, уз изричиту ознаку скраћивања, а подразумевано време извршавања једне команде на 60 s.

На нивоу покретања, агент има највише 40 корака и 1800 s укупног времена.

4.4 ПОСТУПАК ЕВАЛУАЦИЈЕ

4.4.1 Пилот евалуација

Током развоја систем је провераван на 12 малих, синтетички направљених задатака (креирање и измена датотека, исправљање грешака, рефакторисање, рад са тестовима и CSV датотекама). Сваки задатак се извршава у новом контејнеру, а успех се утврђује провером стања система датотека након покретања, а не на основу онога што модел тврди да је урадио. У време пилота систем је имао три алата, па су поређене четири комбинације (`bash`; `bash` и `search`; `bash` и `str_replace`; сва три), са по једним покретањем по задатку, што даје 48 покретања. Алати `read_file` и `find_file` додати су касније, па се услови пилота не поклапају са условима главног експеримента. Резултати су приказани у Табели 2.

Табела 2: Резултати пилот евалуације (12 задатака по услову, једно покретање)

Алати	Успешно	Трошак	Време
`bash`	12/12	$0,0093	73,3 s
`bash`, `search`	12/12	$0,0113	81,1 s
`bash`, `str_replace`	11/12	$0,0080	59,7 s
`bash`, `search`, `str_replace`	10/12	$0,0098	70,3 s

Комбинација `bash` и `str_replace` остварила је најнижи трошак и најкраће време, уз један нерешен задатак, док су услови са алатом `search` били скупљи и спорији. Од три неуспеха у пилоту, један је био пад система због грешке у декодирању излаза команде, која је у међувремену исправљена, а не неуспех модела.

Пошто су ови задаци коришћени за развој и подешавање система, изведени су уз само једно покретање по услову, а разликују се од стварних проблема из репозиторијума и по величини и по сложености, резултати пилота служе само као мотивација за главни експеримент и не представљају доказ за хипотезе.

4.4.2 Скуп података

Главни експеримент користи скуп SWE-bench Lite (Jimenez и др, 2024), у `test` подели, описан у поглављу 3, и то свих 300 задатака из 12 репозиторијума. Највише задатака потиче из репозиторијума `django` (114) и `sympy` (77). Пошто се користи цео скуп, нема избора задатака од стране аутора, па ни пристрасности при избору. Сваки задатак обухвата репозиторијум, ознаку комита пре исправке (`base_commit`) и текст проблема. Скривени тестови (FAIL_TO_PASS и PASS_TO_PASS) агенту нису доступни.

4.4.3 Дизајн експеримента

Експеримент има факторски дизајн 2 × 2 са два фактора: присуство алата `str_replace` и присуство групе алата за претрагу и читање (`search`, `read_file` и `find_file`). Алат `bash` доступан је у свим условима. Комбинацијом нивоа добијају се четири услова приказана у Табели 3.

Табела 3: Услови експеримента

	без `str_replace`	са `str_replace`
без претраге и читања	А: `bash`	Б: `bash`, `str_replace`
са претрагом и читањем	В: `bash`, претрага	Г: сви алати

Сваки задатак пролази кроз сва четири услова, по једном, што даје 1200 покретања. Дизајн је упарен: задатак представља блок, па се услови пореде на истим задацима. Ефекат алата `str_replace` добија се поређењем услова А и Б, као и В и Г, а ефекат групе алата за претрагу и читање поређењем услова А и В, као и Б и Г.

Све остале променљиве су фиксиране и приказане у Табели 4. Посебно, сви услови користе исти системски упит, који не упућује на коришћење ниједног од алата који се пореде нити обесхрабрује одређени начин измене датотека. На тај начин се мери ефекат доступности алата, а не ефекат алата у комбинацији са упутством да се он користи.

Табела 4: Фиксирани параметри експеримента

Параметар	Вредност
модел	`gpt-5.4-nano`
температура	0
режим проширеног закључивања	искључен
највећи број корака	40
највеће трајање покретања	1800 s
приступ мрежи	искључен
број понављања по услову	1
системски упит	идентичан, без упућивања на алате који се пореде

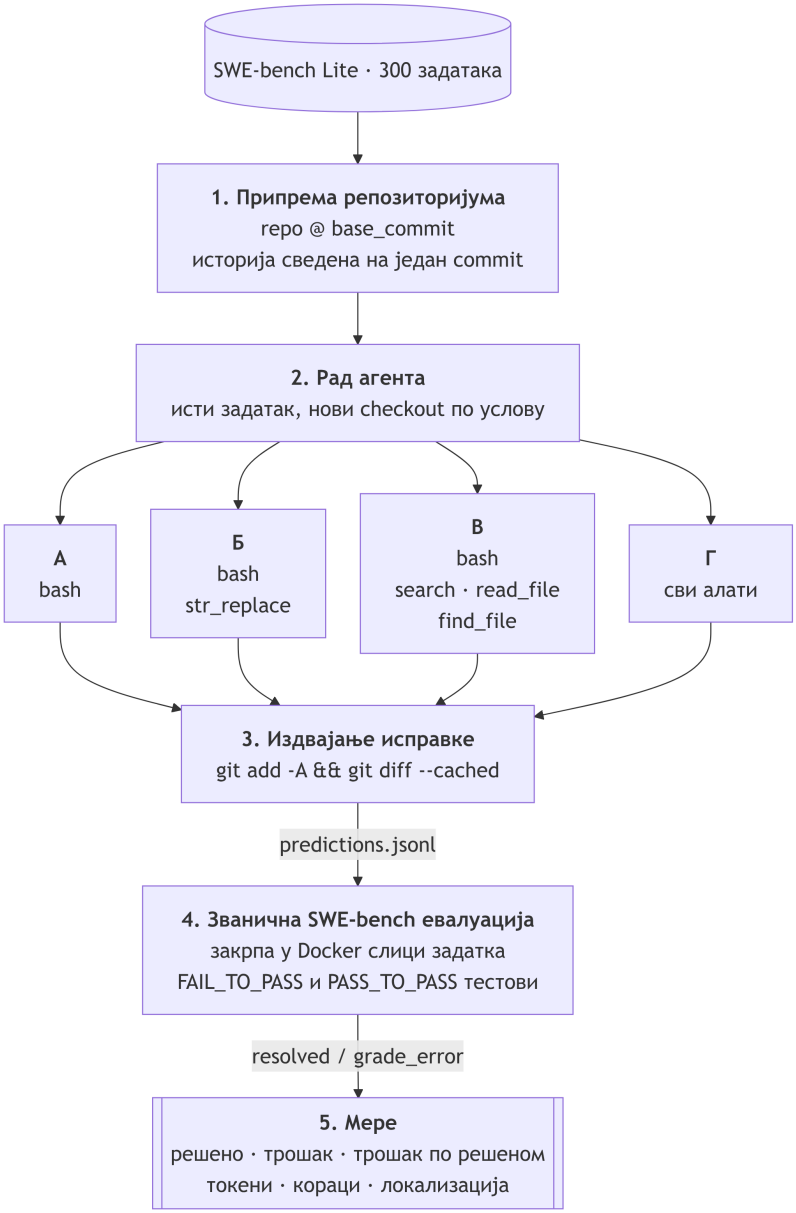
4.4.4 Ток једног покретања

Једно покретање састоји се од четири фазе:

1. **Припрема.** Репозиторијум се из локалне копије поставља на стање комита пре исправке (`base_commit`). Историја верзија се затим брише и замењује једним почетним комитом без удаљеног репозиторијума, тако да агент не може да дође до касније, званичне исправке.
2. **Рад агента.** Агент добија текст проблема као задатак и ради над радном копијом у контејнеру.

3. **Издвајање исправке.** Након завршетка, исправка се добија као разлика радне копије у односу на почетни комит. Бележе се и дијагностички подаци (изостала исправка, број измењених датотека и линија), а исправка се одмах уписује у званичном SWE-bench формату предвиђања.
4. **Оцењивање.** У одвојеној фази, званична SWE-bench процедура примењује исправку у Docker слици припремљеној за тај задатак и покреће скривене тестове. Задатак је решен ако пролазе сви FAIL_TO_PASS и PASS_TO_PASS тестови. Слике окружења за оцењивање преузимају се унапред, јер евалуатор одустаје од преузимања после десет минута и задатак тада добија лажно негативну оцену. Задаци код којих агент није произвео исправку не прослеђују се евалуатору и рачунају се као нерешени. Ако оцена не може да се добије из других разлога, то се бележи засебно и не третира као неуспех агента.

Ток експеримента у целини, од припреме репозиторијума до израчунавања мера, приказан је на Слици 3. Исти задатак пролази кроз сва четири услова, а сваки услов ради над засебном радном копијом.



Слика 3. Ток експеримента

За свако покретање чувају се ред са метрикама, цео разговор са моделом у читљивом облику, необрађене поруке које омогућавају тачно понављање анализе и детаљи евентуалне грешке. Уз сваки ред бележи се и ознака верзије кода система (`harness_sha`). Експеримент је започет пре него што су последње измене кода биле комитоване, па ова ознака указује на претходни комит. Измене које мењају понашање агента унете су пре почетка експеримента, а измене начињене током њега односе се искључиво на чување резултата и наставак прекинутог рада, тако да су сва покретања међусобно упоредива.

4.4.5 Мере

Мере су унапред подељене на три групе, како би се избегло накнадно тражење статистички значајних разлика међу великим бројем колона:

- **Примарне:** стопа решених задатака, трошак по решеном задатку (укупан трошак услова подељен бројем решених задатака у том услову) и стопа локализације, дефинисане у поглављу 3;
- **Секундарне:** трошак по покретању, број улазних и излазних токена, број корака, односно позива модела, број позива алата, укупно време, време у позивима модела и време извршавања алата;
- **Дијагностичке:** изостала исправка, разлог завршетка покретања, грешке алата, број позива по алату, број FAIL_TO_PASS и PASS_TO_PASS тестова који пролазе и величина исправке.

Трошак се рачуна према изразу из поглавља 3, на основу броја токена и јавног ценовника модела `gpt-5.4-nano` (0,20 долара по милиону улазних и 1,25 долара по милиону излазних токена). Стопа локализације рачуна се накнадно, поређењем датотека које мења исправка агента са датотеком коју мења референтна исправка; изостала исправка се рачуна као неуспешна локализација, по узору на Xia и др (2025). Број позива по алату показује да ли је доступан алат заиста коришћен, чиме се ефекат доступности алата разликује од ефекта његовог коришћења.

4.4.6 Правила искључивања

Правила искључивања утврђена су пре увида у резултате:

- одбијање захтева због класификације садржаја (`invalid_prompt`) у примарној анализи сматра се исходом покретања, тј. нерешеним задатком, а стопа одбијања извештава се по услову;
- достизање највећег броја корака или највећег трајања такође је исход (нерешен задатак), а не квар;
- искључују се само покретања прекинута због грешке система или инфраструктуре. Јединица искључивања је задатак: ако је покретање искључено у једном услову, задатак се искључује из свих услова, како

би број задатака по услову остао исти;

- анализа осетљивости понавља поређења само на задацима који су у сва четири услова завршени без грешке API-ја.

4.4.7 Статистичка анализа

Хипотезе Х1 и Х2 проверавају се на главним ефектима факторског дизајна, а хипотеза Х3 поређењем услова Б и Г:

- **Х1 (трошак по решеном задатку).** Упоређује се трошак по решеном задатку услова са алатом `str_replace` (Б и Г) и без њега (А и В). Пошто је ова мера количник збирова, 95% интервал поверења за разлику процењује се бутстреп методом, поновљеним узорковањем задатака са враћањем, при чему се задатак узоркује истовремено у свим условима. Као допунски упарени тест, за сваки задатак рачуна се разлика просечног трошка по покретању са алатом и без њега и примењује Вилкоксонов тест ранга са знаком. Хипотеза се потврђује ако цео интервал поверења лежи испод нуле.
- **Х2 (трошак и локализација).** Разлика у трошку по покретању између услова са групом алата за претрагу и читање (В и Г) и без ње (А и Б) тестира се Вилкоксоновим тестом над разликама по задатку. Разлика у стопи локализације процењује се упареним бутстреп интервалом поверења, уз Мекнемаров тест за појединачна поређења А--В и Б--Г. Део Х2а потврђује се ако је медијана разлике позитивна, а p-вредност после Холмове корекције мања од 0,05, а део Х2б ако цео интервал поверења лежи изнад нуле. Хипотеза Х2 потврђена је само ако важе оба дела.
- **Х3 (неинфериорност).** За разлику стопа решених задатака $R_{Б} - R_{Г}$ рачуна се 95% упарени бутстреп интервал поверења (Efron & Tibshirani, 1993). Неинфериорност се закључује ако је доња граница интервала изнад минус пет процентних поена.

Сви бутстреп интервали добијају се из 10 000 понављања, као перцентилни интервали, са фиксним почетним стањем генератора случајних бројева, па су међусобно упоредиви. Ако у неком бутстреп узорку група нема ниједан решен задатак, трошак по решеном задатку није дефинисан и узорак се одбацује; ако је таквих узорака више

од 5%, хипотеза X1 извештава се само описно.

Примарну фамилију тестова, на коју се примењује Холмова корекција, чине четири теста: Вилкоксонови тестови за X1 и X2a и Мекнемарови тестови стопе локализације за парове А--В и Б--Г. За стопу решених задатака егзактни Мекнемаров тест (McNemar, 1947) примењује се на сва четири контраста нацрта (А--Б, В--Г, А--В и Б--Г), у складу са препоруком да се за упарене податке користе упарени тестови (Dror и др, 2018). Ти тестови чине засебну, секундарну фамилију са сопственом Холмовом корекцијом и не служе за потврду хипотеза. Уз *p*-вредности се увек извештавају величине ефекта и интервали поверења. Интеракција фактора, као и разлике у ефекту по репозиторијуму, величини референтне исправке и броју FAIL_TO_PASS тестова, анализирају се експлоративно.

5 ОПИС И РЕЗУЛТАТИ ЕКСПЕРИМЕНТА

Ово поглавље приказује поставку спроведеног експеримента и добијене резултате. Хипотезе X1 и X3 ослањају се на стопу решених задатака, утврђену званичним SWE-bench евалуатором, док се обе мере хипотезе X2, трошак по покретању и стопа локализације, добијају без покретања тестова. Резултати су приказани редоследом хипотеза, а затим су дискутовани заједно.

5.1 ПОСТАВКА ЕКСПЕРИМЕНТА

Поређена су четири услова добијена укрштањем два фактора: доступности алата `str_replace` за измену датотека и доступности групе алата за претрагу (`search`, `read_file` и `find_file`). Алат `bash` доступан је у свим условима, а услови су приказани у Табели 3.

Експеримент обухвата цео `test` подскуп збирке SWE-bench Lite (300 задатака) у сва четири услова, дакле $300 \times 4 = 1200$ покретања, са једним покретањем по услову. Агент ради у контејнеру без приступа мрежи, са највише 40 корака и ограничењем од 1800 секунди по задатку. Укупан обрачунати трошак покретања обухваћених примарном анализом износи $32,20, о чему се детаљније говори међу ограничењима рада. Укупно утрошено време износи 14,3 сата за та покретања, односно 14,7 сати за свих 1200.

Након оцењивања, анализом транскрипата утврђено је да прва верзија алата `search` није подржавала глоб образац са путањом, због чега је око половине позива претраге у условима В и Г враћало грешку уместо резултата (49,5% позива у услову В и 56,0% у услову Г), а бројач грешака алата то није бележио. Алат је исправљен, а услови В и Г поновљени су у целости, са истим нацртом, истим задацима и истим поступком анализе. Услови А и Б нису поновљени, јер не користе алате за претрагу, а њихов део система није мењан. Резултати који следе односе се на ту комбинацију покретања, а првобитне вредности за услове В и Г наведене су у одељку 5.2.5.

У једном покретању услова В позив модела прекинут је истеком времена на страни провајдера. По правилима искључивања реч је о инфраструктурном квару, па је цео задатак искључен из свих услова. Примарна анализа стога обухвата 299 задатака по услову, а анализа осетљивости 258 задатака.

Оцењивање решености спроведено је након завршетка свих покретања, на засебном рачунару са 64 GB радне меморије, без поновног покретања агента и без додатних позива модела. Слике окружења за све задатке, укупно приближно 100 GB, преузете су пре почетка оцењивања, а само оцењивање спроведено је са осам паралелних процеса. У малом броју покретања евалуатор није покренуо тестове, јер је исправка агента у корен репозиторијума додала замене за спољашње модуле (`mpmath` и `distutils`), које заклањају праве модуле. Ова покретања су неуспех агента и рачунају се као нерешена.

5.2 РЕЗУЛТАТИ

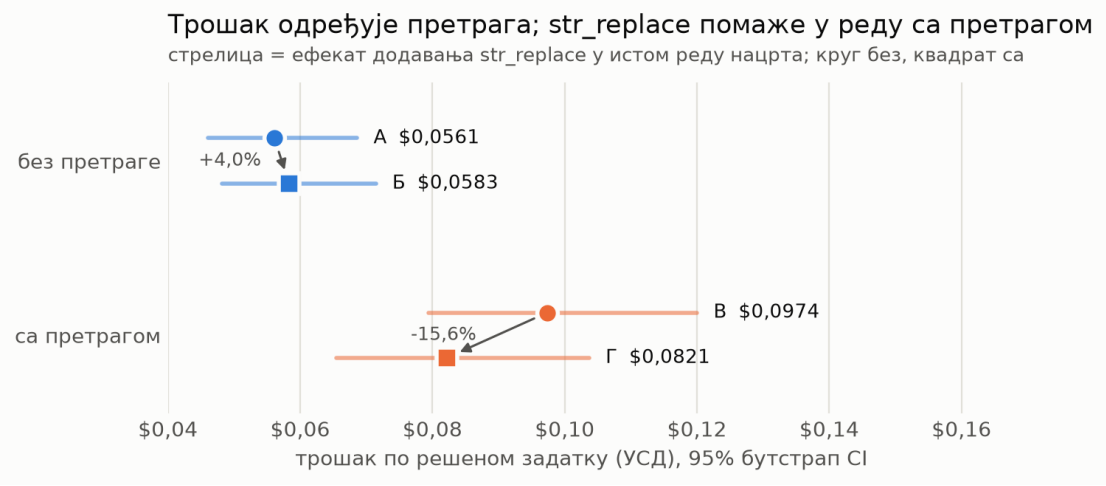
5.2.1 Провера хипотезе Х1

Хипотеза Х1 тврди да доступност алата `str_replace` смањује трошак по решеном задатку. Трошак по решеном задатку дефинисан је као однос укупног трошка свих покретања у групи и броја решених задатака у тој групи, чиме се у меру урачунава и трошак неуспешних покретања. Поређен је главни ефекат фактора, група услова са алатом (Б и Г) наспрам групе услова без њега (А и В), при чему свака група садржи по један услов са алатима за претрагу и један без њих. Стопе решених задатака и трошкови по условима приказани су у Табели 5, а трошак по решеном задатку са

интервалима поверења и на Слици 4.

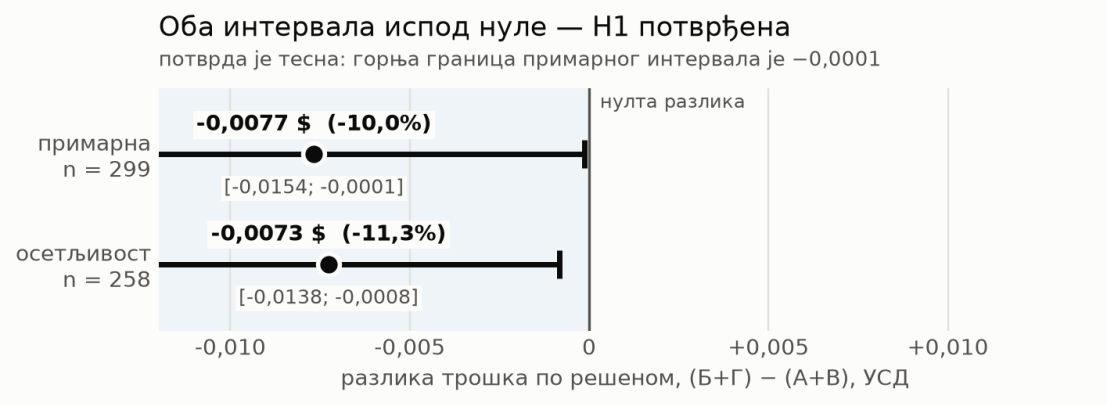
Табела 5: Трошак по решеном задатку по условима (299 задатака по услову)

Услов	Алати	Решено	Трошак	Трошак/покр.	Трошак/решен
А	`bash`	111 (37,1%)	$6,23	$0,0208	$0,0561
Б	+ `str_replace`	119 (39,8%)	$6,94	$0,0232	$0,0583
В	+ претрага	112 (37,5%)	$10,90	$0,0365	$0,0974
Г	сви алати	99 (33,1%)	$8,13	$0,0272	$0,0821

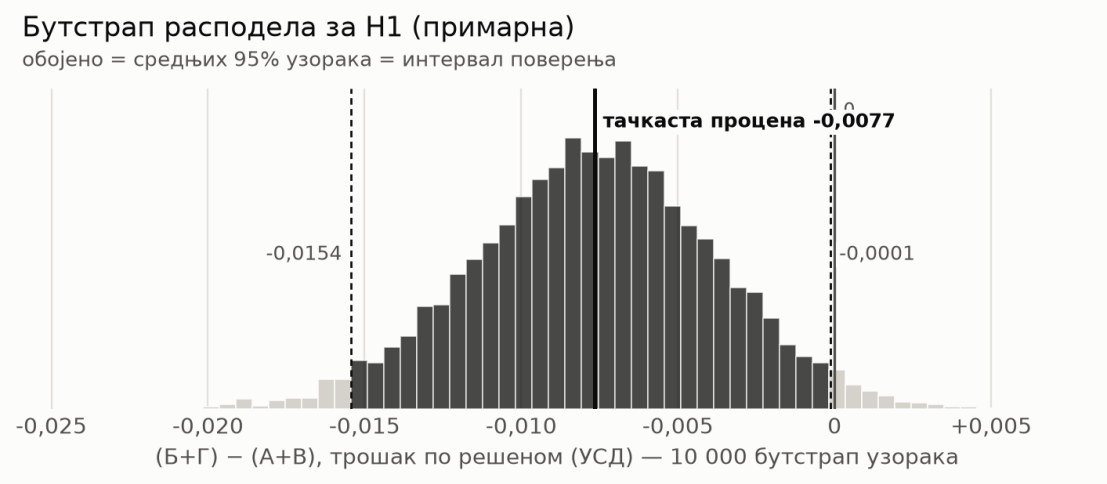


Слика 4. Трошак по решеном задатку по условима, са 95% бутстреп интервалима поверења

X1 --- потврђена, гранично. Трошак по решеном задатку у групи са алатом `str_replace` износи $0,0691 ($15,07 за 218 решених задатака), а у групи без њега $0,0768 ($17,13 за 223 решена). Разлика износи −$0,0077 (−10,0%), са упареним бутстреп интервалом поверења од 95% [−$0,0154, −$0,0001] (10 000 понављања, поновно узорковање задатака; ниједан узорак није одбачен због групе без решених задатака). Критеријум потврде био је да цео интервал лежи испод нуле и он јесте испуњен, али његова горња граница лежи непосредно уз нулу, а испод нуле налази се 97,8% бутстреп узорака, па се налаз извештава као граничан. Интервали поверења за примарну анализу и анализу осетљивости приказани су на Слици 5, а бутстреп расподела разлике на Слици 6.

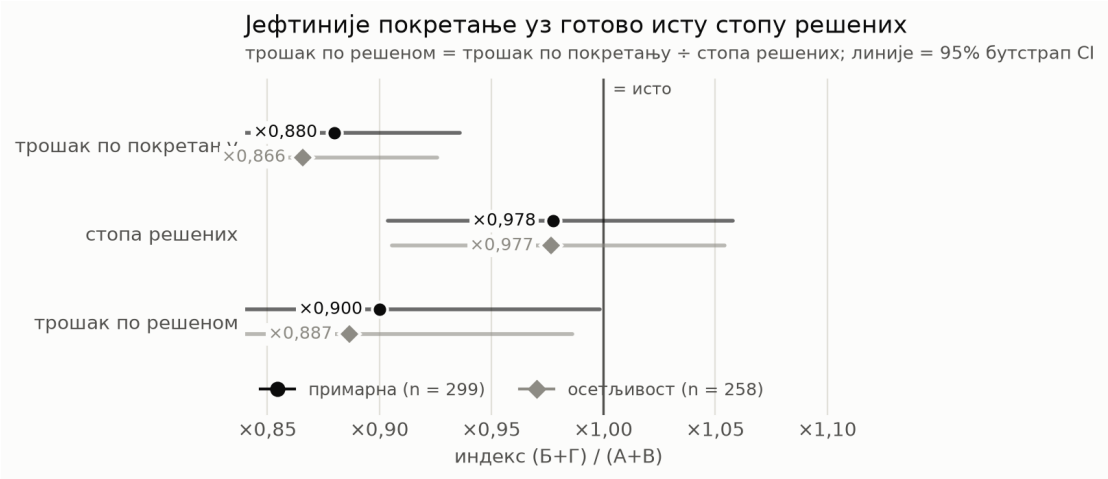


Слика 5. Интервали поверења за разлику трошка по решеном задатку између услова са алатом `str_replace` и услова без њега



Слика 6. Бутстреп расподела разлике трошка по решеном задатку у примарној анализи

Допунски Вилкоксонов тест упарених рангова, спроведен над разликом трошка по покретању, даје медијану разлике −$0,00288 и $p = 2{,}91 \cdot 10^{-8}$, при чему је разлика негативна у 205 од 299 задатака, и опстаје после Холмове корекције примарне фамилије. Покретања са алатом `str_replace` јефтинија су за 12,0% по покретању, док је стопа решених задатака у тој групи нижа за 2,2% (36,5% наспрам 37,3%), па од уштеде по покретању на нивоу решеног задатка остаје смањење од 10,0%. Ово разлагање приказано је на Слици 7.



Слика 7. Разлагање трошка по решеном задатку на трошак по покретању и стопу решених задатака

Анализа осетљивости, ограничена на 258 задатака на којима ни у једном услову није дошло до одбијања захтева од стране провајдера, даје исти закључак: разлика износи −$0,0073, са интервалом [−$0,0138, −$0,0008], који такође цео лежи испод нуле.

Главни ефекат, међутим, није једнак у оба реда нацрта. У реду без алата за претрагу прелаз са услова А на услов Б не мења трошак по решеном задатку (+$0,0022, 95% интервал поверења [−$0,0062, +$0,0105]), док у реду са претрагом прелаз са услова В на услов Г тај трошак смањује (−$0,0152, интервал [−$0,0299, −$0,0003]). Интервал поверења за разлику ова два ефекта не обухвата нулу (−$0,0175, интервал [−$0,0350, −$0,0001]), али је поређење редова унапред означено као експлоративно, па се наводи као образац, а не као утврђена интеракција. Ефекат на коме почива потврда хипотезе Х1 тиме потиче из услова са алатима за претрагу, а не из услова без њих.

5.2.2 Мера локализације

Као примарна мера за хипотезу Х2 користи се локализација, која не зависи од покретања тестова: покретање се сматра локализованим ако скуп датотека које мења исправка агента и скуп датотека које мења референтна исправка имају непразан пресек. Изостала исправка се по дефиницији сматра нелокализованом. Мера је укључена у план анализе пре првог израчунавања.

У Табели 6 приказани су резултати по условима, а у Табели 7 исходи покретања.

Табела 6: Резултати по условима (299 задатака по услову)

Услов	Алати	Локализација	Трошак/покр.	Корака	Ул. токена
А	`bash`	78,3%	$0,0208	16,0	94 683
Б	+ `str_replace`	77,6%	$0,0232	15,7	107 486
В	+ претрага	77,3%	$0,0365	18,8	173 639
Г	сви алати	76,6%	$0,0272	15,0	130 135

Табела 7: Исходи покретања по условима

Услов	Алати	Изостала исправка	`invalid_prompt`	Исцрпљени кораци
А	`bash`	22	23	4
Б	+ `str_replace`	31	29	6
В	+ претрага	24	1	26
Г	сви алати	21	2	18

Однос између просечног трошка по покретању и стопе локализације приказан је на Слици 8. Услови са алатима за претрагу налазе се десно, али не и више: скупљи су, а стопа локализације им није већа.



Слика 8. Однос трошка по покретању и стопе локализације по условима

5.2.3 Провера хипотезе Х2

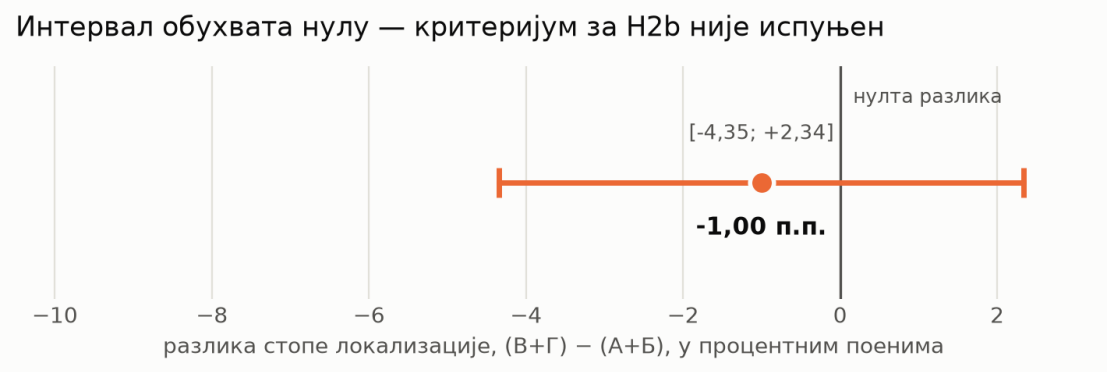
Хипотеза Х2 тврди да доступност групе алата за претрагу повећава трошак по покретању (Х2а), али истовремено повећава стопу локализације (Х2б). Према унапред утврђеном плану, Х2 се сматра потврђеном само ако важе и Х2а и Х2б.

Х2а --- потврђена. Поређен је главни ефекат групе за претрагу, услови В и Г наспрам услова А и Б, над упареним разликама по задатку: за сваки задатак израчуната је разлика просечног трошка у условима са претрагом и просечног трошка у условима без ње. Медијана разлике износи +$0,0030 (95% интервал поверења [+$0,0019, +$0,0053]), а просек +$0,0098, при чему је разлика позитивна на 194 од 299 задатака. Вилкоксонов тест упарених рангова даје $W = 11\,068$, $p = 3{,}19 \cdot 10^{-14}$ и величину ефекта $r_{rb} = +0{,}506$, па остаје значајан и после Холмове корекције примарне фамилије.

Мера трошка рачуна се по ценовнику над свим улазним токенима, без попуста за кеширани улаз, па представља горњу границу стварног трошка, што је размотрено међу ограничењима рада. Пошто је таква мера детерминистичка функција броја токена, поређење услова њоме није изложено утицају кеширања.

Х2б --- одбачена. Разлика стопе локализације између услова са претрагом и услова без ње износи −1,00 процентни поен, са упареним бутстреп интервалом поверења од 95% [−4,35, +2,34] процентних поена (10 000 понављања, поновно узорковање задатака). Критеријум потврде био је да цео интервал лежи изнад нуле, а добијени интервал обухвата нулу, па се део Х2б одбацује: алати за претрагу стопу локализације нити повећавају нити смањују. Допунски Мекнемарови егзактни тестови сагласни су са тим налазом и ниједан није близу прага значајности: за пар А--В $b = 22$, $c = 19$ и $p = 0{,}755$, а за пар Б--Г $b = 25$, $c = 22$ и $p = 0{,}771$. Ни један не опстаје после Холмове корекције примарне фамилије тестова.

Овај резултат приказан је и на Слици 9, где се види да интервал поверења обухвата нулу.



Слика 9. Упарени бутстреп интервал поверења за разлику стопе локализације између услова са претрагом и услова без ње

Пошто Х2б није потврђена, хипотеза Х2 се одбацује. Део Х2а понаша се како је предвиђено: алати за претрагу повећавају трошак по покретању. Део Х2б изостаје: очекивана корист не постоји, јер се стопа локализације између услова са претрагом и услова без ње не разликује. Налаз, дакле, не показује штету од алата за претрагу, него одсуство користи уз већи трошак.

5.2.4 Провера хипотезе Х3

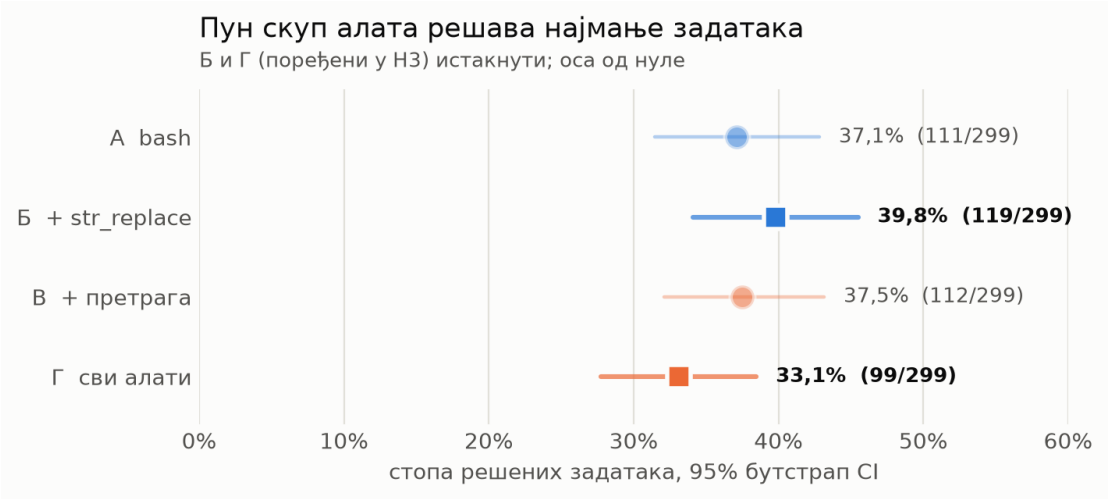
Хипотеза Х3 формулисана је као тврдња о неинфериорности: стопа решених задатака у услову Б (`bash` и `str_replace`) није нижа од стопе у услову Г (сви алати) за више од пет процентних поена. Маргина је утврђена пре оцењивања решености. Критеријум потврде јесте да доња граница упареног бутстреп интервала поверења разлике $R_{Б} - R_{Г}$ лежи изнад минус пет процентних поена. Упарени исходи ова два услова приказани су у Табели 8.

Табела 8: Упарени исходи услова Б и Г (299 задатака)

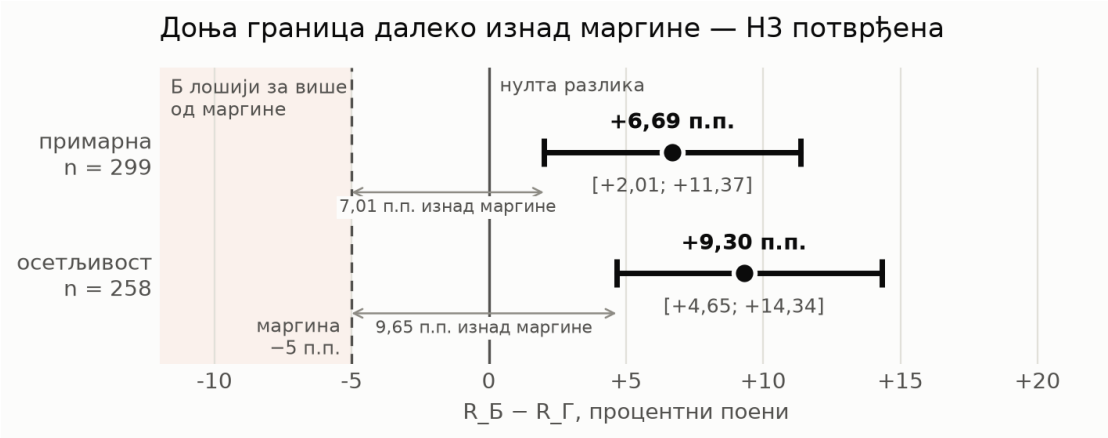
	Г решио	Г није решио	Укупно
Б решио	83	36	119
Б није решио	16	164	180
Укупно	99	200	299

Х3 --- потврђена. Услов Б решио је 119 од 299 задатака (39,8%, 95% интервал поверења [34,1, 45,5]%), а услов Г 99 задатака (33,1%, интервал [27,8, 38,5]%). Разлика износи +6,69 процентних поена, са упареним бутстреп интервалом поверења од 95% [+2,01, +11,37] процентних поена (10 000 понављања, поновно узорковање

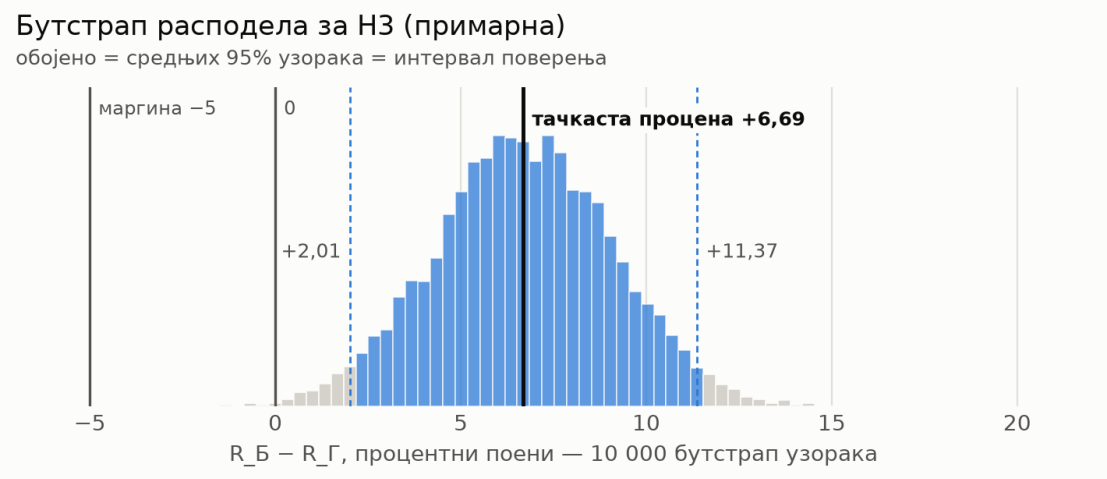
задатака). Доња граница интервала лежи 7,01 процентних поена изнад маргине неинфериорности. Апсолутне стопе решених задатака далеко су од нуле, па потврда неинфериорности није последица уског интервала при ниским стопама. Стопе решених задатака по условима приказане су на Слици 10, интервал поверења разлике на Слици 11, а његова бутстреп расподела на Слици 12.



Слика 10. Стопе решених задатака по условима, са 95% бутстреп интервалима поверења



Слика 11. Интервали поверења за разлику стопа решених задатака између услова Б и Г у односу на маргину неинфериорности



Слика 12. Бутстреп расподела разлике стопа решених задатака између услова Б и Г у примарној анализи

Интервал у целини лежи изнад нуле, што указује да услов Б решава више задатака од услова Г. Супериорност није била унапред утврђена хипотеза, али је у складу са унапред утврђеним секундарним поређењем: егзактни Мекнемаров тест за пар Б--Г даје $b = 36$, $c = 16$ и $p = 7{,}8 \cdot 10^{-3}$, и опстаје после Холмове корекције фамилије од четири контраста нацрта. Ниједан други контраст не опстаје после корекције: пару В--Г одговара $p = 0{,}0789$, а пару А--Б $p = 0{,}256$.

Анализа осетљивости, ограничена на 258 задатака на којима ни у једном услову није дошло до одбијања захтева од стране провајдера, потврђује закључак: разлика износи +9,30 процентних поена, са интервалом [+4,65, +14,34].

5.2.5 Утицај дефекта у алату `search`

Резултати приказани у овом поглављу добијени су после исправке дефекта у алату `search`, описаног у одељку о поставци експеримента. Пошто су првобитне вредности за услове В и Г ушле у ранију обраду, у Табели 9 упоређена су два покретања истих услова, са истим моделом, истим задацима и истим системским упутством, која се разликују само по томе да ли алат `search` подржава глоб образац са путањом.

Табела 9: Услови В и Г пре и после исправке алата `search` (пре исправке 300 задатака, после исправке 299)

Мера	В пре	В после	Г пре	Г после
Решено	105 (35,0%)	112 (37,5%)	84 (28,0%)	99 (33,1%)
Локализација	74,3%	77,3%	71,7%	76,6%
Исцрпљени кораци	41	26	54	18
Изостала исправка	38	24	52	21
Корака по задатку	23,8	18,8	22,4	15,0
Трошак по решеном	$0,1006	$0,0974	$0,1111	$0,0821

Исправка једног дефекта мења слику о алатима за претрагу. Удео покретања која исцрпе дозвољени број корака пада са 18,0% на 6,0% у услову Г и са 13,7% на 8,7% у услову В, а број изосталих исправки са 52 на 21, односно са 38 на 24. Са тим се мења и исход хипотезе Х1, која је на првобитним вредностима била одбачена (+$0,0024, интервал [−$0,0058, +$0,0110]). Исходи хипотеза Х2 и Х3 остају исти, али са мањим разликама: разлика стопе локализације опада са −4,67 на −1,00 процентни поен, а разлика стопа решених задатака између услова Б и Г са +11,67 на +6,69 процентних поена. Мера „ефекта алата“ тиме обухвата и квалитет његове имплементације, што је размотрено у дискусији и међу ограничењима рада.

5.3 ДИСКУСИЈА РЕЗУЛТАТА

Потврда хипотезе Х1 не потиче од тога што алат `str_replace` штеди излазне токене. Алат ради оно за шта је намењен: измена се задаје навођењем сегмента текста уместо поновним исписивањем датотеке или састављањем команде, па број излазних токена опада, при чему медијана упарене разлике између услова Б и А износи приближно 150 токена по задатку. Излазни токени, међутим, чине мање од 10% трошка. Преостали део отпада на улазне токене, јер се у сваком позиву модела шаље цела историја разговора, па трошак одређују количина прочитаног кода и број корака, а не величина измене, која обично заузима један до два корака на крају покретања.

Насупрот малој уштеди стоји фиксни трошак: декларација алата, која по мерењу на моделу додаје 223 улазна токена, шаље се у сваком позиву модела, што на око 16 корака износи више од 3 500 улазних токена по покретању, неколико пута више од уштеде на излазу. Због тога алат у условима без претраге трошак не мења, а цео главни

ефекат потиче из реда нацрта са алатима за претрагу. Модел при том алат не користи доследно: у 102 покретања услова Б није га позвао ниједном. Упарена разлика трошка по покретању између услова Б и А није статистички значајна (медијана +$0,0004, Вилкоксон $p = 0{,}10$), па алат `str_replace` у условима без претраге трошак ни не смањује ни значајно не повећава. Главни ефекат од −12,0% по покретању у целости потиче из реда нацрта са алатима за претрагу, где исти алат покретање скраћује са 18,8 на 15,0 корака и појефтињује га за приближно четвртину.

Удруживањем услова прикрива се различито понашање алата у два реда нацрта. У условима без алата за претрагу, при прелазу са услова А на Б, трошак по решеном задатку готово је непромењен ($0,0561 наспрам $0,0583). У условима са претрагом, при прелазу са В на Г, покретања постају знатно јефтинија ($0,0365 наспрам $0,0272 по покретању) и краћа (18,8 наспрам 15,0 корака), а стопа решених пада са 37,5% на 33,1%, што је мање од уштеде, па трошак по решеном задатку опада за 15,7% ($0,0974 наспрам $0,0821). Интервал поверења за разлику ова два ефекта не обухвата нулу (−$0,0175, 95% интервал поверења [−$0,0350, −$0,0001]), али је поређење редова нацрта унапред означено као експлоративно, па се образац наводи као налаз те врсте, а не као утврђена интеракција.

Трошак по решеном задатку и даље знатно више зависи од алата за претрагу него од алата `str_replace`: услови без претраге решавају задатак за $0,056--0,058, а услови са претрагом за $0,082--0,097, дакле приближно 1,5 пута скупље. Са становишта цене, избор скупа алата за претрагу важнији је од избора алата за измену.

Резултат за хипотезу Х2 супротан је полазној претпоставци да наменски алати за претрагу помажу агенту да пронађе место грешке: они трошак повећавају, а локализацију не побољшавају. Механизам се назире у Табелама 6 и 7: услови са претрагом троше приближно 1,5 пута више улазних токена, а у услову В просечан број корака расте са приближно 16 на 18,8. Број покретања која исцрпе дозвољени број корака и даље је већи у условима са претрагом (26 у услову В и 18 у услову Г наспрам 4--6 без ње), али се тај губитак више не преводи у изостанак исправке: изостала исправка чешћа је у условима без претраге (22 у услову А и 31 у услову Б) него у условима са њом (24 и 21). Додатни трошак претраге, дакле, не одлази на покретања која никада не стигну до измене.

Раст броја корака не описује у потпуности колико алата модел позове, јер позив алата није исто што и корак: модел у једном кораку може да позове више алата, што се у условима са претрагом дешава чешће, а последњи корак често нема позив алата, јер модел само завршава одговором. Просечан број позива алата по задатку износи 15,5 у услову А, 15,8 у услову Б, 20,6 у услову В и 16,7 у услову Г: у услову В модел, дакле, у просеку упути 20,6 позива кроз 18,8 корака, док у условима без претраге на корак долази приближно један позив.

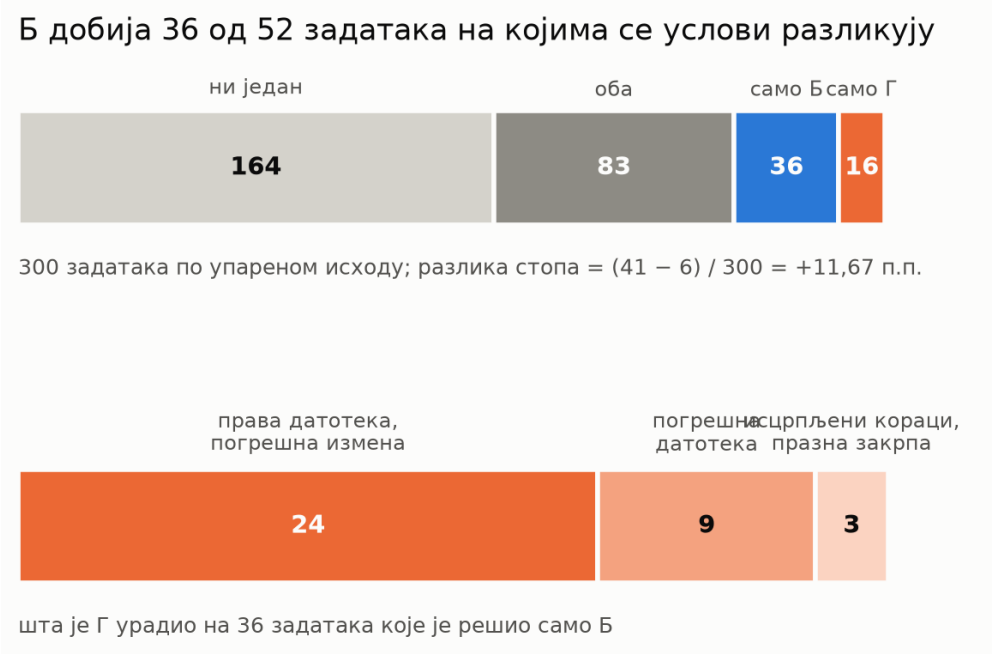
Овај налаз је у складу са резултатима N. F. Liu и др (2024), по којима модели слабије користе информације у средини дугачког контекста, као и са тезом Xia и др (2025) да сложенији систем не доноси нужно бољи резултат. Истовремено, он одступа од налаза J. Yang и др (2024a), где је прилагођени интерфејс, укључујући и претрагу, побољшао успешност у односу на рад само са љуском. Разлика може потицати од величине модела, пошто је овде коришћен мали модел, коме дужи контекст и већи број могућих акција више отежавају него што помажу, као и од имплементације алата за претрагу, која је размотрена у одељку 5.2.5 и међу ограничењима рада. Вреди приметити да сама претрага не смањује стопу решених задатака: услов В решава практично исто колико и услов А (37,5% наспрам 37,1%). Пад се јавља само у услову Г, дакле тек у комбинацији оба фактора.

Разлика између услова А и Б није статистички разлучива на мери локализације ($b = 17$, $c = 15$, $p = 0{,}86$). То није доказ да алат `str_replace` нема ефекта, него последица тога што мера не може да га обухвати: тај алат делује у фази измене датотеке, након што је место грешке већ пронађено, док локализација мери искључиво проналажење. Вредност тог алата проверава се хипотезом Х1, која се ослања на број решених задатака.

Поређење по задацима показује колико се услови заправо разликују: од 299 задатака, сва четири услова локализују истих 195 задатака, ниједан услов не локализује 31 задатак, а на преосталих 73 задатка исходи се разликују. Услови се, дакле, слажу на три четвртине скупа, па разлике у стопи локализације потичу од мање четвртине задатака.

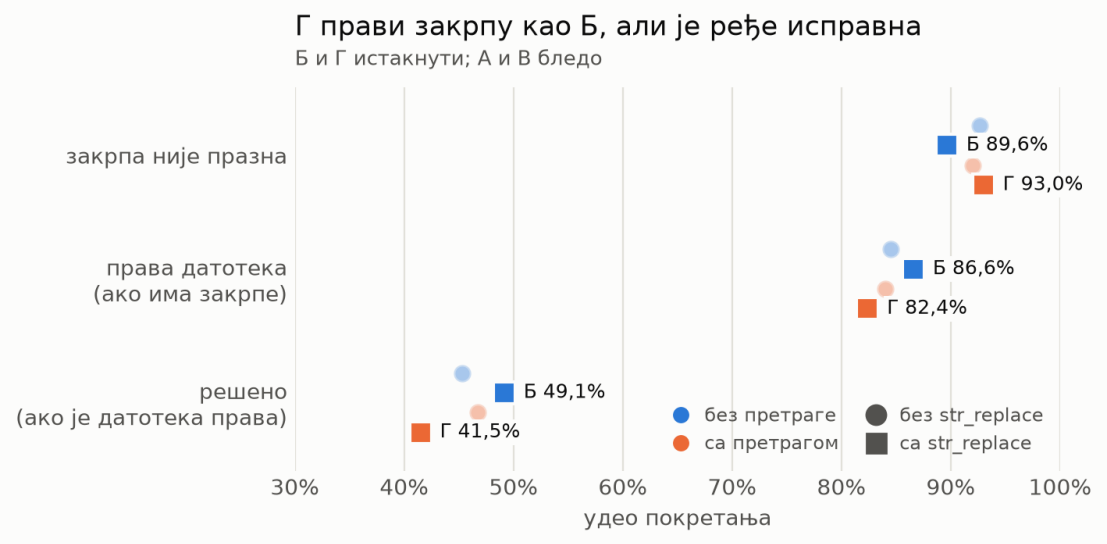
Резултат за хипотезу Х3 најјачи је налаз експеримента: најмањи скуп алата који садржи наменски алат за измену не само да не заостаје за пуним скупом, него решава

приближно 20% више задатака (119 наспрам 99). Од 52 задатка на којима се исходи ова два услова разликују, услов Б је успешнији на 36. Упарени исходи и исход услова Г на задацима које је решио само услов Б приказани су на Слици 13.



Слика 13. Упарени исходи услова Б и Г и исходи услова Г на задацима које је решио само услов Б

Разлагање стопе решених задатака на узастопне фазе показује где услов Г губи: не у томе што не произведе исправку, него у томе што је исправка чешће нетачна. Изостала исправка више није извор губитка: услов Г производи исправку у 93,0% покретања, дакле чешће него услов Б (89,6%). Разлика настаје касније. Када произведе исправку, услов Г нешто ређе погађа одговарајућу датотеку (82,4% наспрам 86,6%), а највећи део разлике долази из квалитета измене: међу покретањима која мењају одговарајућу датотеку, услов Б решава 49,1% задатака, а услов Г 41,5%. Услов Г, дакле, исправку производи најчешће од свих услова, а она је најређе тачна. На 36 задатака које је решио само услов Б, агент у услову Г је у 24 случаја изменио одговарајућу датотеку, али погрешно, у девет случајева изменио погрешну датотеку, а у три исцрпео дозвољени број корака. Фазе нису строго узастопне, јер је мали број покретања решен без измене референтне датотеке, што на поређење не утиче. Разлагање по фазама за сва четири услова приказано је на Слици 14.



Слика 14. Разлагање стопе решених задатака на узастопне фазе по условима

Експлоративна анализа транскрипата показује да агент у услову Г ређе проверава претпоставку покретањем кода, а више чита. Просечан број покретања кода по задатку дат је у Табели 10, а исти подаци приказани су и на Слици 15. Услов Г покреће код најређе од свих услова, а разлика је највећа пре прве измене, дакле у фази репродукције грешке. Код се може покренути само кроз алат `bash`.

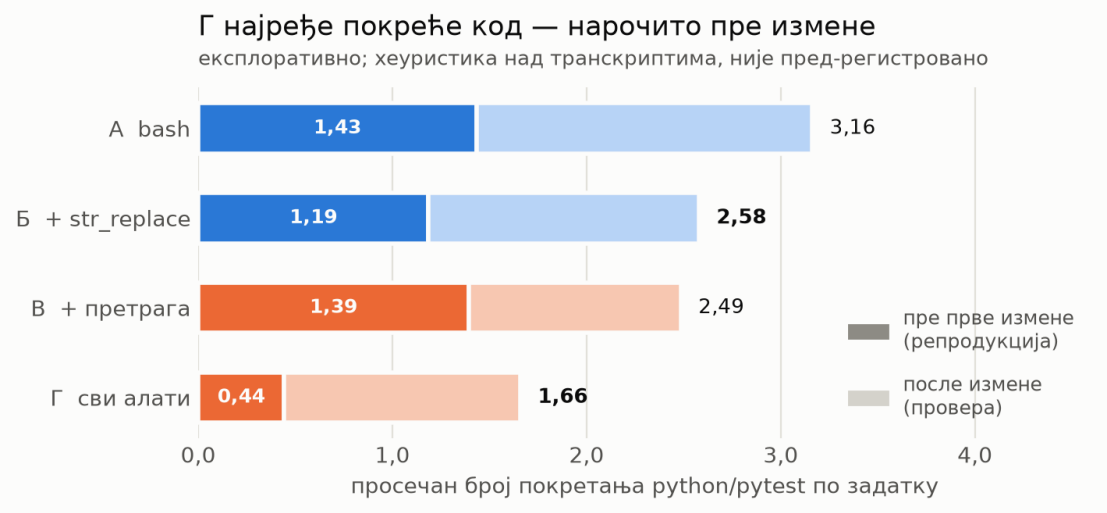
Табела 10: Просечан број покретања кода по задатку

Фаза	А	Б	В	Г
Пре прве измене (репродукција)	1,43	1,19	1,39	0,44
Укупно по задатку	3,16	2,58	2,49	1,66

Услов В, који има исту групу алата за претрагу, репродукује грешку скоро три пута чешће од услова Г, а на одговарајућој датотеци решава 46,8% задатака, приближно као услови А и Б. Уз то, услов Г по задатку чита највише контекста међу условима са алатом за измену (130 135 улазних токена наспрам 107 486 у услову Б), а притом прави најмање корака (15,0 наспрам 15,7): више прочитаног не долази кроз више корака, него кроз више контекста по кораку.

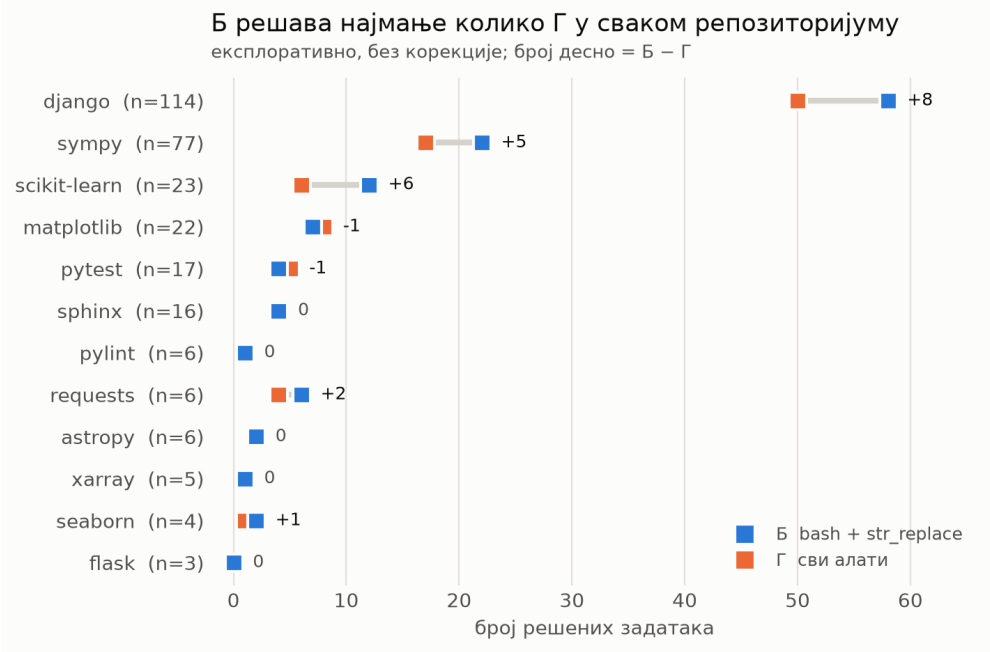
Тај образац указује да алати за читање и претрагу померају агента ка тумачењу кода уместо ка провери претпоставке, па он до одговарајуће датотеке стиже приближно подједнако успешно, али чешће упише погрешну измену. Пад је, према томе, повезан са комбинацијом оба фактора, што је у складу са секундарним поређењима, у којима је значајан само пар Б--Г. Бројање покретања кода засновано је на хеуристичкој анализи

команди и није унапред утврђено, део тих покретања завршава се грешком због недостајућих модула, а разлика у стопи решених постоји и међу покретањима која су код покренула, па се ова разлика не може сматрати узроком пада, већ само пратећим обрасцем.



Слика 15. Просечан број покретања кода пре и после прве измене по условима

Предност услова Б није равномерна по репозиторијумима. Услов Б решио је више задатака у пет репозиторијума (`django` 58 наспрам 50, `sympy` 22 наспрам 17, `scikit-learn` 12 наспрам 6, `requests` 6 наспрам 4 и `seaborn` 2 наспрам 1), услов Г у два (`matplotlib` 8 наспрам 7 и `pytest` 5 наспрам 4), док су у преосталих пет изједначени. Поређење по репозиторијумима је експлоративно и без корекције за вишеструко тестирање, а исходи по репозиторијумима приказани су на Слици 16. Пошто су покретања прекинута одбијањем захтева знатно чешћа у услову Б (29 наспрам два), а у примарној анализи се рачунају као нерешена, примарни резултат је у том погледу конзервативан: искључивањем тих задатака разлика се повећава.



Слика 16. Број решених задатака у условима Б и Г по репозиторијумима

Поређење две верзије истог услова показује да мера „ефекта алата“ обухвата и квалитет његове имплементације. Са истим моделом, истим задацима и истим скупом алата, исправка једног дефекта подигла је стопу решених задатака у услову Г са 28,0% на 33,1%, смањила удео покретања која исцрпе дозвољени број корака са 18,0% на 6,0% и променила исход хипотезе X1. Закључци о скуповима алата зато важе за дату имплементацију, што је у складу са налазом J. Yang и др (2024a) да нацрт интерфејса између агента и рачунара битно утиче на успешност.

Посматрани заједно, резултати делимично подржавају полазну тезу рада. Од три хипотезе потврђене су X1, гранично, и X3, док је X2 одбачена. Практична препорука ослања се на X3: комбинација алата `bash` и `str_replace` решава највише задатака од свих услова (39,8%), уз трошак по решеном задатку који се статистички не разликује од најјефтинијег услова A ($0,0583 наспрам $0,0561). Други део тезе није потврђен, јер пун скуп алата не доноси предност при лоцирању грешке, а по трошку по решеном задатку остаје скупљи од мањих скупова.

5.4 ОГРАНИЧЕЊА

Резултате рада треба тумачити у светлу следећих ограничења, груписаних у четири целине. Три су најважнија: експеримент је спроведен са једним моделом, свака

конфигурација покренута је само једном, а агент није могао да покреће тестове пројекта.

Модел и скуп података. Ова ограничења одређују на шта се резултати смеју уопштити.

- **Један модел.** Експеримент је спроведен само са моделом `gpt-5.4-nano`, па се резултати не могу непосредно уопштити на јаче моделе. Смер тог утицаја не може се унапред претпоставити: очекивање да слабији модели имају више користи од помоћних алата није потврђено, јер је пун скуп алата, у овој имплементацији, малом моделу одмогао. Посебно, није проверено да ли се разлика између услова Б и Г одржава код модела веће способности, који би алате за претрагу могли користити делотворније.
- **Природа скупа SWE-bench Lite.** Скуп садржи искључиво самосталне исправке у једној датотеци са највише три измењена блока, што је тип измене у коме `str_replace` има највећу предност. Код измена кроз више датотека ефекат може бити мањи.
- **Квалитет скупа података.** Део задатака садржи решење у опису проблема, а део тестова није довољан да открије нетачне исправке (Aleithan и др, 2024). Постоји и ризик да су задаци били део података за тренирање модела. Резултати зато важе у оквиру бенчмарка.
- **Заштитни механизми провајдера.** Модел је коришћен преко Azure OpenAI сервиса, чији заштитни механизми (енг. *guardrails*) проверавају садржај сваког захтева. Пошто су захтеви садржали изворни код и излазе команди, класификатор је део њих означавао као неприхватљив садржај, иако то нису били, и враћао грешку `invalid_prompt`. Понављање захтева отклонило је део ових случајева, али не све, па одређени број задатака није могао да буде решен из разлога који не зависе ни од модела ни од скупа алата. Стопа таквих грешака извештава се по услову.

Мере. Ова ограничења односе се на то шта употребљене мере обухватају, а шта не.

- **Мера локализације није мера исправности.** Локализација утврђује само да исправка агента дира бар једну датотеку коју дира и референтна исправка,

не и да је измена исправна. Закључак о хипотези X2 стога важи за фазу проналажења места грешке, док се исправност проверава хипотезама X1 и X3, на основу резултата званичног евалуатора.

- **Мера трошка не обухвата кеширање.** Трошак се рачуна по ценовнику над свим улазним токенима, без попуста за кеширани улаз, па представља горњу границу стварног трошка; број кешираних токена није бележен током извођења, те се попуст не може накнадно применити. Улазни токени чине највећи део обрачуна, а горња граница трошка свих покретања обухваћених примарном анализом износи $32,20; уз потпуно кеширање улаза тај износ би био $4,97, а стварни рачун провајдера налазио се близу те доње границе. Пошто је употребљена мера детерминистичка функција броја токена, поређење услова њоме није изложено утицају кеширања. Ипак, услови са претрагом имају дуже путање, па и вероватно виши удео кеширања, услед чега би разлика у стварном трошку могла бити мања од пријављене. Мера је изабрана зато што је поновљива и независна од провајдера, за разлику од стварне стопе кеширања, која зависи од распореда и времена извођења.
- **Трошак по решеном задатку је однос.** Пошто трошак по решеном задатку не постоји за појединачни нерешен задатак, мера се не може поставити у упарени тест по задатку. Интервал поверења добијен је бутстреп методом, а допунски Вилкоксонов тест односи се на трошак по покретању, који није иста мера. Однос збирова осетљив је на мали број скупих покретања, што је при 218 и 223 решена задатка по групи ублажено, али не и отклоњено.
- **Маргина неинфериорности задата је одлуком аутора.** Вредност од пет процентних поена није изведена из спољашњег стандарда, већ представља процену губитка који би био практично занемарљив у односу на смањење сложености скупа алата. Маргина је утврђена пре оцењивања решености. Добијени резултат није осетљив на њен избор, јер доња граница интервала лежи изнад нуле, тако да би закључак важио за сваку ненегативну маргину.

Имплементација алата и одступања од плана. Измерени ефекат приписује се скупу алата, али зависи и од тога како су алати написани и како је експеримент изведен.

- **Агент ради „наслепо“.** Због јединствене слике окружења, описане у поглављу 4, агент не може да покрене тестове пројекта, а покретање кода пројекта често се завршава грешком због недостајућих модула, па се ослања претежно на читање кода и опис проблема. Агент је у појединим покретањима то покушавао да заобиђе додавањем замена за недостајуће модуле у радну копију. То је значајно одступање од упоредивих система, попут SWE-agent (J. Yang и др, 2024a) и Agentless (Xia и др, 2025), који агенту омогућавају покретање тестова и проверу исправке, па је стопа решених задатака вероватно нижа него што би иначе била. Одлука важи подједнако у свим условима, тако да не утиче на поређење услова.
- **Резултат важи за ову имплементацију алата.** Измерени ефекат приписује се скупу алата, али зависи и од тога како је сваки алат написан. Прва верзија алата `search` није подржавала глоб обрасце са путањом (нпр. `sympy/**/*.py`), које је модел често користио, па је 49,5% позива претраге у услову B и 56,0% у услову Г враћало грешку уместо резултата, при чему бројач грешака алата то није бележио. После исправке тог једног дефекта услов Г решава петнаест задатака више, а исход хипотезе X1 се мења, што је приказано у одељку 5.2.5. Поред тога, `search` враћа највише 50 погодака без могућности страничења, без редова контекста око поготка и без ознаке да је излаз одсечен, због чега модел не разликује потпун од непотпуног резултата и мора да допуни увид додатним позивима. Другачији нацрт исте функционалности могао би дати другачији исход, па закључак треба читати као тврдњу о овој површини алата, а не о алатима за претрагу уопште.
- **Ситнији чиниоци на страни алата.** Позивање алата је научена способност (Schick и др, 2023), па зависи и од тога колико су назив и шема понуђеног алата слични онима које је модел видео током обуке; на то указује и покушај модела да кроз `bash` покрене `apply_patch`, алат за измену датотека који у овом систему не постоји, и то у 280 покретања услова А, 185 услова Б, 277 услова B и 17 услова Г. Уз то, декларација алата `bash` захтева поље у коме модел пре сваког позива образлаже поступак, док остали алати такво поље немају, па образлагање пада неравномерно по условима (приближно 291 токен по задатку у услову А и 266 у услову Б, наспрам 105 у услову B и 33 у услову Г), пошто се

и сам `bash` користи неједнако (сви позиви алата у услову А, 94,9% у услову Б, 26,7% у услову В и 13,7% у услову Г). Удео образлагања у укупној потрошњи је мали (испод 0,5%), а смер ниједног од ова два чиниоца није једнозначан нити мерљив овим нацртом. Оба су у складу са налазом J. Yang и др (2024a) да дизајн интерфејса битно утиче на успешност агента.

- **Одступање од закљученог плана анализе.** План је предвиђао по једно покретање сва четири услова. Пошто је после оцењивања откривен дефект у алату `search`, услови В и Г поновљени су у целости са исправљеним алатом, а услови А и Б преузети су из првог покретања, јер не користе алате за претрагу и њихов део система није мењан. Резултати се стога односе на комбинацију два покретања између којих је протекло два дана.Хипотезе, мере, маргина неинфериорности и поступак анализе нису мењани, а првобитне вредности за услове В и Г приказане су у одељку 5.2.5.

Недостајући подаци и поновљивост. Ова ограничења односе се на покретања која нису дала употребљив исход и на немогућност процене варијабилности.

- **Једно покретање по услову.** Није могуће проценити варијабилност резултата истог задатка у истом услову, а излаз модела није потпуно детерминистички ни при температури 0.
- **Недостајући подаци нису независни од услова.** Одбијања описана међу заштитним механизмима провајдера нису равномерно распоређена: у условима без алата за претрагу било их је 52 (23 у услову А и 29 у услову Б), а у условима са претрагом свега три (једно у услову В и два у услову Г). Разлог је то што агент без наменских алата више необрађеног кода пропушта кроз команде љуске. Недостајући подаци тиме корелирају са независном променљивом, па се њихова стопа извештава по услову, а налази се проверавају анализом осетљивости на задацима који су завршени у сва четири услова. У примарној анализи ова покретања улазе као исход услова: трошак који су до тренутка одбијања произвела урачунава се, а задатак није решен, што оптерећује трошак по решеном задатку у условима без претраге. Анализа осетљивости на 258 задатака завршених у сва четири услова даје исти закључак за све три хипотезе. За X1 разлика износи −$0,0073 (95% интервал поверења

[−\$0,0138, −\$0,0008]), дакле хипотеза остаје потврђена; за X3 разлика стопа решених задатака износи +9,30 процентних поена (интервал [+4,65, +14,34]), такође потврђена. За X2 остаје потврђен само део X2а: медијана разлике трошка по покретању износи +\$0,0020 (интервал [+\$0,0006, +\$0,0034]), уз $W = 9887$, $p = 1{,}3 \cdot 10^{-8}$ и величину ефекта $r_{rb} = +0{,}408$, док се део X2б и на овом подскупу одбацује, и то израженије него у примарној анализи, јер је стопа локализације у условима са претрагом нижа за 3,68 процентних поена, са интервалом [−7,17, −0,39] који цео лежи испод нуле. Хипотеза X2 се, дакле, одбацује и у анализи осетљивости.

- **Исходи без покретања тестова.** У малом броју покретања евалуатор није покренуо тестове, јер је исправка агента додала замене за спољашње модуле које заклањају праве модуле. Ова покретања рачунају се као нерешена, што одговара стварном исходу, јер исправка у том облику не може да прође тестове. Покретања без иједног извршеног теста има 23 (седам у услову А, три у услову Б, седам у услову В и шест у услову Г), што је 1,9% свих покретања обухваћених примарном анализом. У 12 од њих евалуатор пријављује грешку увоза модула, што одговара описаном узроку, а у преосталих 11 исправка агента онемогућава прикупљање тестова на други начин, најчешће синтаксном грешком или изменом великог броја датотека.

6 ЗАКЉУЧАК

Предмет овог рада било је питање у којој мери скуп алата који агент за програмирање има на располагању утиче на његову успешност и трошак, независно од избора језичког модела. Питање је испитано на практичној имплементацији агента и на целом `test` подскупу збирке SWE-bench Lite, у нацрту 2 × 2 који раздваја два фактора: наменски алат за измену датотека (`str_replace`) и групу алата за претрагу и читање кода (`search`, `read_file` и `find_file`). Сва четири услова покренута су над истих 300 задатака, са истим моделом и истим системским упутством, а хипотезе, маргина неинфериорности и план статистичке анализе утврђени су пре оцењивања решености. Услови са алатима за претрагу поновљени су у целости пошто је у алату `search` откривен дефект, а примарна анализа обухвата 299 задатака по услову.

6.1 ГЛАВНИ НАЛАЗИ

Резултати не потврђују интуицију да агент са богатијим скупом алата ради боље. Најмањи скуп алата који садржи наменски алат за измену, `bash` и `str_replace`, решио је 119 од 299 задатака (39,8%), а пун скуп алата 99 (33,1%). Разлика од 6,7 процентних поена, са 95% интервалом поверења [+2,0, +11,4], потврђује хипотезу X3 о неинфериорности мањег скупа и указује да мањи скуп решава више задатака. Мањи скуп је истовремено и јефтинији: трошак по решеном задатку износи $0,058 наспрам $0,082.

Хипотеза X2, према којој алати за претрагу повећавају трошак, али побољшавају проналажење места грешке, потврђена је само у првом делу. Услови са претрагом троше знатно више (медијана упарене разлике +$0,0098 по покретању, приближно 1,5 пута више улазних токена), али стопу локализације не мењају: разлика износи −1,0 процентни поен, са 95% интервалом поверења [−4,3, +2,3], који обухвата нулу. Додатни трошак прати већи број покретања која исцрпе дозвољени буџет корака: 4--6 у условима без претраге наспрам 26 у услову В и 18 у услову Г.

Хипотеза X1, према којој `str_replace` смањује трошак по решеном задатку, потврђена је гранично: разлика износи −10,0%, уз интервал поверења [−$0,0154, −$0,0001], чија горња граница лежи непосредно уз нулу. Ефекат при том не потиче од уштеде на излазним токенима, који чине мање од 10% трошка, него готово у целости из реда нацрта са алатима за претрагу, где алат покретање скраћује и појефтињује. У реду без алата за претрагу трошак по решеном задатку остаје непромењен, јер га одређује количина прочитаног контекста, који се шаље у сваком позиву модела, а не величина измене, која заузима један до два корака на крају рада.

6.2 ТУМАЧЕЊЕ

Резултати указују на исти образац: за мали модел додатни алати за претрагу имају цену која није садржана у самом алату. Са њима агент троши више улазних токена, чешће исцрпљује дозвољени број корака и другачије ради. У пуном скупу алата агент најређе покреће код, нарочито пре прве измене (0,44 наспрам 1,19 пута по задатку у услову Б). Место грешке при томе проналази приближно једнако успешно, али га ређе исправно

мења: међу покретањима која мењају одговарајућу датотеку, пун скуп алата решава 41,5% задатака, а услов Б 49,1%. Ове разлике утврђене су експлоративно и не могу се сматрати узроком пада, али су у складу са чињеницом да ниједан од два фактора засебно не доводи до значајног пада, већ тек њихова комбинација.

Шире посматрано, резултати указују да се избор алата не може свести на питање да ли алат ради оно што му је намена. Утицај алата на агента одређују структура трошка, промена понашања коју изазивају и детаљи њихове имплементације. Те ствари не виде се из описа алата, већ тек из мерења.

6.3 ПРАКТИЧНЕ ПРЕПОРУКЕ

За агента заснованог на малом моделу, резултати упућују на следеће:

- почети од минималног скупа алата (`bash` и алат за измену) и сваки додатни алат уводити тек када мерење покаже корист;
- трошак процењивати по решеном задатку, а не по покретању, и уз њега пратити број корака и удео покретања која исцрпе буџет;
- оптимизацију трошка усмерити на количину контекста који агент чита, а не на излаз модела, јер улазни токени чине више од 90% трошка;
- при увођењу новог алата пратити у транскриптима како га агент стварно користи и шта му алат враћа, а не само збирне бројаче;
- алат за претрагу пројектовати тако да подржава глоб обрасце са путањом, страничење и контекст око поготка, јер детаљ те врсте мења измерени ефекат целог скупа алата;
- грешке алата мерити независно од тога како их алат сам пријављује, јер бројач уграђен у алат може да прећути управо оне грешке које највише коштају.

6.4 ОГРАНИЧЕЊА

Закључци важе у границама нацрта. Сви услови покренути су са једним, малим моделом, при температури 0 и са једним покретањем по задатку, па није проверено да ли се образац одржава код модела веће способности, који би алате за претрагу могли користити другачије; смер тог утицаја не може се унапред претпоставити. Измерени

ефекат односи се на ову имплементацију алата, а не на алате за претрагу уопште. Трошак је рачунат без попуста за кеширани улаз, па представља горњу границу стварног трошка. Део покретања прекинут је одбијањем захтева од стране провајдера, неравномерно по условима, а анализа осетљивости која те задатке искључује даје исте закључке за хипотезе X1 и X3. Услови са алатима за претрагу поновљени су после исправке дефекта у том алату, па се резултати односе на комбинацију два покретања. Механизам описан у тумачењу утврђен је експлоративном анализом транскрипата и није био део унапред утврђеног плана.

6.5 ДАЉИ РАД

Најнепосреднији наставак јесте понављање експеримента са моделом веће способности, макар на подскупу задатака, чиме би се проверило да ли је налаз својство малог модела или скупа алата. Поновљена покретања по задатку омогућила би да се процени колики део неслагања између услова потиче од случајности. Посебно је занимљиво испитати да ли се губитак пуног скупа алата може отклонити променом нацрта самих алата, на пример страничењем и контекстом око погодака у претрази, или изричитим подстицањем агента да грешку репродукује пре измене. Коначно, мерење стварне стопе кеширања и укључивање алата за покретање тестова уз приступ мрежи проширили би слику о томе како скуп алата утиче на однос цене и успешности.

УПОТРЕБА АЛАТА ЗАСНОВАНИХ НА ВЕШТАЧКОЈ ИНТЕЛИГЕНЦИЈИ

У изради овог рада коришћени су алати засновани на великим језичким моделима, и то на два начина.

Прво, велики језички модел представља предмет истраживања: агентски систем развијен за потребе рада користи модел `gpt-5.4-nano`, а сви резултати приказани у раду добијени су покретањем тог система, како је описано у поглављима 4 и 5.

Друго, при писању рада коришћен је асистент заснован на великом језичком моделу, и то за обликовање и језичко дотеривање текста, проналажење и проверу библиографских података, као и за помоћ при обликовању документа у систему LaTeX. Идеја рада, имплементација агентског система, поставка и извођење експеримента, обрада резултата и закључци представљају самосталан рад аутора. Аутор је проверио све податке, тврдње и референце наведене у раду и преузима одговорност за њихову тачност.

ЛИТЕРАТУРА

Aleithan, R., Xue, H., Mohajer, M. M., Nnorom, E., Uddin, G., & Wang, S. (2024). SWE-Bench+: Enhanced Coding Benchmark for LLMs. https://arxiv.org/abs/2410.06992

Austin, J., Odena, A., и др. (2021). Program Synthesis with Large Language Models. https://arxiv.org/abs/2108.07732

Bowman, S. R., & Dahl, G. E. (2021). What Will it Take to Fix Benchmarking in Natural Language Understanding? *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. https://arxiv.org/abs/2104.02145

Brown, T. B., и др. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2005.14165

Card, D., Henderson, P., Khandelwal, U., Jia, R., Mahowald, K., & Jurafsky, D. (2020). With Little Power Comes Great Responsibility. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://arxiv.org/abs/2010.06595

Chen, M., и др. (2021). Evaluating Large Language Models Trained on Code. https://arxiv.org/abs/2107.03374

Dror, R., Baumer, G., Shlomov, S., & Reichart, R. (2018). The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1383–1392. https://aclanthology.org/P18-1128/

Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.

Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., & Stoica, I. (2024). LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. https://arxiv.org/abs/2403.07974

Ji, Z., и др. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, *55*(12). https://arxiv.org/abs/2202.03629

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2310.06770

Lin, Z., Zhu, J., Zhou, M., Wang, X., Sun, Z., и др. (2026). To Run or Not to Run: Analyzing the Cost-Effectiveness of Code Execution in LLM-Based Program Repair. *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. https://arxiv.org/abs/2606.26978

Liu, J., Xia, C. S., Wang, Y., & Zhang, L. (2023). Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2305.01210

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, *12*, 157–173. https://arxiv.org/abs/2307.03172

McNemar, Q. (1947). Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, *12*(2), 153–157. https://doi.org/10.1007/BF02295996

Ouyang, L., и др. (2022). Training Language Models to Follow Instructions with Human Feedback. *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2203.02155

Pan, J., Wang, X., Neubig, G., Jaitly, N., Ji, H., Suhr, A., & Zhang, Y. (2025). Training Software Engineering Agents and Verifiers with SWE-Gym. *International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2412.21139

Patil, S. G., Zhang, T., Wang, X., & Stoica, I. (2023). Gorilla: Large Language Model Connected with Massive APIs. https://arxiv.org/abs/2305.15334

Phan, H. N., Nguyen, T. N., Nguyen, P. X., & Bui, N. D. Q. (2024). HyperAgent: Generalist Software Engineering Agents to Solve Coding Tasks at Scale. https://arxiv.org/abs/2409.16299

Qin, Y., Liang, S., Ye, Y., и др. (2023). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. https://arxiv.org/abs/2307.16789

Reimers, N., & Gurevych, I. (2017). Reporting Score Distributions Makes a Difference: Performance Study of LSTM-networks for Sequence Tagging. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://arxiv.org/abs/1707.09861

Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4. издање). Pearson.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use

Tools. *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2302.04761

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/1706.03762

Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., и др. (2025). OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2407.16741

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2201.11903

Wei, Y., Duchenne, O., Copet, J., Carbonneaux, Q., Zhang, L., Fried, D., Synnaeve, G., Singh, R., & Wang, S. I. (2025). SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution. *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2502.18449

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., и др. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. https://arxiv.org/abs/2308.08155

Xia, C. S., Deng, Y., Dunn, S., & Zhang, L. (2025). Agentless: Demystifying LLM-based Software Engineering Agents. *Proceedings of the ACM on Software Engineering (FSE)*. https://arxiv.org/abs/2407.01489

Xiao, Y.-A., Gao, P., Peng, C., & Xiong, Y. (2025). Reducing Cost of LLM Agents with Trajectory Reduction. https://arxiv.org/abs/2509.23586

Xie, C., Li, B., Gao, C., Du, H., Lam, W., Zou, D., & Chen, K. (2025). SWE-Fixer: Training Open-Source LLMs for Effective and Efficient GitHub Issue Resolution. *Findings of the Association for Computational Linguistics (ACL)*. https://arxiv.org/abs/2501.05040

Yang, H., Yu, Q., & Desell, T. (2026). When Does Restricting a Coding Agent to `execute_code` Help? A Regime × Agent-Design Ablation [Radionica Agentic Software Engineering (SE 3.0), KDD 2026]. https://arxiv.org/abs/2607.10569

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024a). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2405.15793

Yang, J., Jimenez, C. E., Zhang, A. L., Lieret, K., Yang, J., и др. (2024b). SWE-bench Multimodal: Do AI Systems Generalize to Visual Software Domains? https://arxiv.org/abs/2410.03859

Yang, J., Lieret, K., Jimenez, C. E., Wettig, A., Khandpur, K., и др. (2025). SWE-smith: Scaling Data for Software Engineering Agents. https://arxiv.org/abs/2504.21798

Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2024). τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. https://arxiv.org/abs/2406.12045

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2210.03629

Zan, D., Huang, Z., Liu, W., Chen, H., Zhang, L., и др. (2025). Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. https://arxiv.org/abs/2504.02605

Zhang, J., Zhang, Y., & Huang, Y. (2026). Coupling Planning with Episodic Memory in LLM Agents for Software Issue Resolution. https://arxiv.org/abs/2608.06811

Zhang, Y., Ruan, H., Fan, Z., & Roychoudhury, A. (2024). AutoCodeRover: Autonomous Program Improvement. https://arxiv.org/abs/2404.05427

Zhuo, T. Y., Vu, M. C., Chim, J., Hu, H., Yu, W., и др. (2024). BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2406.15877